

Technische Hochschule Deggendorf

Fakultät Angewandte Informatik

Studiengang Bachelor Cyber-Security

Kurs: Security Engineering – SS 2026

Projektdokumentation

Secure DevOps (DevSecOps) – Sicherheitstechnische Betrachtung automatisierter CI/CD-Pipelines

Vorgelegt von:

Christof Renner (22301943)

Manuel Friedl (12306626)

Arbeitsaufteilung: je 50 %

Betreuer:

Prof. Dr. Martin Schramm

Datum: 30. Juli 2026

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation und Problemstellung	4
1.2. Zielsetzung	4
1.3. Abgrenzung	4
1.4. Methodik	4
1.5. Aufbau der Arbeit	5
1.6. Methodisches Vorgehen	5
2. Grundlagen	5
2.1. DevSecOps	5
2.2. Software Supply Chain Security	6
2.3. Frameworks	6
2.4. Security Engineering Framework	6
2.5. Zugriffskontroll- und Integritätsmodelle	6
3. System unter Betrachtung	8
3.1. Architekturüberblick	8
3.2. Schützenswerte Assets	8
3.3. Trust Boundaries	9
3.4. Angreifermodell	9
4. Bedrohungsanalyse (STRIDE)	10
4.1. STRIDE-Mapping pro Pipeline-Stufe	10
4.2. Threat Walkthroughs	10
4.3. Vertiefung: Poisoned Pipeline Execution (PPE)	12
5. Gegenmaßnahmen und Implementierung	14
5.1. Source-Stufe	14
5.2. Build-Stufe	15
5.3. Test- und Scan-Stufe	15
5.4. Package- und Sign-Stufe	15
5.5. SBOM-Lebenszyklus	16
5.6. Deploy-Stufe	18
5.7. Auszug aus der CI-Implementierung	18
5.8. Mapping Bedrohung → Maßnahme	19
5.9. Fallstudie: <code>tj-actions/changed-files</code> (März 2025)	19
6. Evaluation	20
6.1. SLSA-Einstufung	20
6.2. OpenSSF Scorecard	22
6.3. Wirksamkeitsnachweis durch Negativtests	22
6.4. Coverage der STRIDE-Kategorien	23
6.5. Validierung gegen OWASP Top 10 CI/CD	23
6.6. KPIs und Sicherheitsmetriken	24
6.7. Compliance und regulatorischer Rahmen	25
6.8. Restrisiken	27
6.9. Zielkonflikt Geschwindigkeit vs. Assurance	27

6.10. Faktor Mensch und Anreize	28
7. Fazit und Ausblick	30
A. Vollständige Pipeline-Artefakte	31
A.1. CI-Workflow (.github/workflows/ci.yml)	31
A.2. CD-Workflow (.github/workflows/cd.yml)	36
A.3. Scorecard-Workflow (.github/workflows/scorecard.yml)	39
A.4. Dockerfile (Multi-Stage, Non-Root)	40
A.5. .github/CODEOWNERS	41
A.6. Dependabot (.github/dependabot.yml)	41
A.7. Abhängigkeits-Definition (app/requirements.in)	42

1. Einleitung

1.1. Motivation und Problemstellung

Moderne Softwareentwicklung basiert auf hochgradig automatisierten Build-, Test- und Deployment-Prozessen. CI/CD-Pipelines sind tief in cloud-native Infrastrukturen integriert und verfügen über weitreichende Zugriffsrechte auf sensible Ressourcen wie Quellcode-Repositories, Secrets, Container-Registries und Produktionsumgebungen. Angriffe auf Softwarelieferketten – etwa SolarWinds (2020), Codecov (2021), Log4Shell (2021) und der 3CX-Vorfall (2023) – belegen, dass der Entwicklungsprozess selbst zu einem primären Angriffsziel geworden ist.

Aus Sicht des Security Engineering ist eine Pipeline weniger ein Tooling-Problem als ein *Systemproblem*: Sie verbindet menschliche Identitäten, externe Abhängigkeiten, Build-Hosts, Registries und Deployment-Targets über mehrere Vertrauensgrenzen hinweg. Eine isolierte Betrachtung einzelner Werkzeuge greift daher zu kurz.

1.2. Zielsetzung

Ziel der Arbeit ist eine systematische sicherheitstechnische Betrachtung einer repräsentativen CI/CD-Pipeline. Konkret werden:

- schützenswerte Assets und Trust Boundaries identifiziert,
- Bedrohungen je Pipeline-Stufe nach STRIDE strukturiert analysiert,
- geeignete Sicherheitsmechanismen abgeleitet und in einer prototypischen Pipeline (GitHub Actions) umgesetzt,
- die Wirksamkeit dieser Maßnahmen anhand von SLSA-Levels und OpenSSF-Scorecard evaluiert,
- verbleibende Restrisiken transparent gemacht.

1.3. Abgrenzung

Nicht Gegenstand der Arbeit sind: vollständige Härtung des Cloud-Tenants, Penetrationstests der Pipeline-Plattform selbst sowie organisatorische Maßnahmen jenseits der Branch-Protection-Konfiguration. Die Pipeline wird auf GitHub Actions umgesetzt; die Übertragbarkeit auf GitLab CI wird nicht im Detail untersucht und in Kapitel 7 lediglich als weiterführende Forschungsrichtung benannt.

1.4. Methodik

Die Untersuchung folgt einem strukturierten, an das Security-Engineering-Framework angelehnten Vorgehen. Zunächst werden System, Assets und Trust Boundaries modelliert (Kap. 3). Darauf aufbauend werden Bedrohungen je Pipeline-Stufe systematisch nach STRIDE erhoben und mit dem Angreifermodell sowie – zur Plausibilisierung – mit den

OWASP-Top-10-CI/CD-Risiken trianguliert (Kap. 4). Jeder Bedrohung wird anschließend eine konkrete, im Prototyp umgesetzte Gegenmaßnahme zugeordnet (Kap. 5). Die Risikoeinstufung ist qualitativ (niedrig/mittel/hoch) und kombiniert im Sinne der Vorlesungsdefinition die Eintrittswahrscheinlichkeit (*Gefahr*) mit dem Schadensausmaß (*Gefährdung*) zum resultierenden *Risiko*. Die abschließende Bewertung (Kap. 6) stützt sich nicht auf Selbstauskunft allein, sondern wird durch externe, automatisierte Werkzeuge gegengeprüft – OpenSSF Scorecard für die Repository-Posture sowie *cosign* und *slsa-verifier* für Signatur und Provenance.

1.5. Aufbau der Arbeit

Kapitel 2 klärt die Begriffe DevSecOps, Supply-Chain-Security und die zugrundeliegenden Frameworks (STRIDE, SLSA, SSDF) sowie die klassischen Zugriffskontroll- und Integritätsmodelle. Kapitel 3 beschreibt das modellierte System und seine Trust Boundaries. Kapitel 4 enthält das STRIDE-Threat-Model. Kapitel 5 ordnet jeder Bedrohung konkrete Gegenmaßnahmen zu und dokumentiert die Implementierung im Prototyp. Kapitel 6 bewertet die Wirksamkeit, Kapitel 7 fasst zusammen und diskutiert Restrisiken.

1.6. Methodisches Vorgehen

Die Arbeit kombiniert eine analytische mit einer konstruktiven Phase und ist durchgängig am Vier-Quadranten-Modell des Security Engineering (Abschnitt 2.4) ausgerichtet. Zunächst werden Assets, Trust Boundaries und ein Angreifermodell bestimmt (Kapitel 3). Daraus wird das Bedrohungsmodell nach STRIDE je Pipeline-Stufe abgeleitet (Kapitel 4) und gegen die OWASP-Top-10-CI/CD-Risiken (Abschnitt 6.5) trianguliert; reale Vorfälle (SolarWinds, Codecov, *tj-actions*) dienen als Plausibilitätsanker. Die abgeleiteten Maßnahmen werden in einem lauffähigen GitHub-Actions-Prototyp umgesetzt (Kapitel 5; vollständige Artefakte in Anhang A). Die Evaluation erfolgt zweistufig: Eine SLSA-Selbsteinstufung wird durch die extern und automatisiert erhobene OpenSSF-Scorecard gegengeprüft (Kapitel 6).

2. Grundlagen

2.1. DevSecOps

DevSecOps bezeichnet die Integration sicherheitstechnischer Aktivitäten in den DevOps-Lebenszyklus mit dem Ziel, Sicherheit als kontinuierlichen, automatisierten Bestandteil von Build und Deployment zu etablieren („shift left“ und „shift right“). Sicherheitsmaßnahmen werden nicht nachgelagert aufgesetzt, sondern als Teil derselben Pipeline ausgeführt, die auch das Artefakt produziert. DevSecOps lässt sich dabei als kontinuierliche, automatisierte Ausprägung des klassischen *Security Development Lifecycle* (SDL) verstehen, dessen Praktiken im NIST SSDF (Abschnitt 6.7) prozessual kodifiziert sind.

2.2. Software Supply Chain Security

Eine Software Supply Chain umfasst alle Personen, Prozesse, Werkzeuge und Abhängigkeiten, die zur Erstellung und Auslieferung eines Artefakts beitragen. Angriffsklassen nach [1] sind insbesondere:

- **Source**-Angriffe (z. B. kompromittierte Maintainer-Accounts, Typosquatting),
- **Build**-Angriffe (z. B. kompromittierte Runner, manipulierte Compiler/Plugins),
- **Dependency**-Angriffe (z. B. Übernahme verwaister Pakete),
- **Distribution**-Angriffe (z. B. Image-Tag-Replacement in Registries).

2.3. Frameworks

STRIDE

Microsoft-Methodik zur Bedrohungsmodellierung mit den Kategorien *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, *Elevation of Privilege*.

SLSA v1.0

Supply-chain Levels for Software Artifacts definiert Anforderungen an Build-Integrität in vier Stufen (Build L1–L3, Source L1–L3).

NIST SSDF (SP 800-218)

Secure Software Development Framework – Praktiken über den gesamten SDLC.

OpenSSF Scorecard

Automatisierte Bewertung von Repository-Posture anhand objektiver Checks.

2.4. Security Engineering Framework

Die Arbeit folgt dem in der Lehrveranstaltung eingeführten Vier-Quadranten-Modell des Security Engineering nach Anderson: *Policy*, *Mechanism*, *Assurance* und *Incentives*. Tabelle 1 ordnet jedem Quadranten konkrete Bestandteile der untersuchten Pipeline zu; auf den Quadranten *Incentives* wird in Abschnitt 6.10 vertiefend eingegangen.

2.5. Zugriffskontroll- und Integritätsmodelle

Die in der Lehrveranstaltung behandelten klassischen Zugriffskontroll- und mehrstufigen Sicherheitsmodelle wurden ursprünglich für Betriebssysteme und militärische Systeme formuliert, lassen sich aber unmittelbar auf eine CI/CD-Pipeline übertragen: Eine Pipeline ist ein Mehr-Subjekt-System mit abgestuften Vertrauens- und Integritätsniveaus und muss genau jene Eigenschaften durchsetzen, die diese Modelle formalisieren.

Zugriffskontrollmodelle. Die *Zugriffskontrollmatrix* ordnet jedem Paar (Subjekt, Objekt) die erlaubten Operationen zu; praktisch wird sie spaltenweise als *Access Control List*

Quadrant	Umsetzung in der Pipeline
Policy	Branch-Protection-Regeln, CODEOWNERS, Required Status Checks, Coverage-Schwelle 80 %, HIGH/CRITICAL als Build-Breaker, Manual-Approval-Gate für Production.
Mechanism	Bandit, CodeQL, pip-audit, Trivy, Syft (SBOM), Cosign Keyless Signing, SLSA-Provenance, OIDC-Federation, Action-Pinning auf Commit-SHA.
Assurance	SLSA-Level-Bewertung, OpenSSF Scorecard (wöchentlich), SARIF-Upload in Code-Scanning, Cosign-Verify im CD-Workflow, Sigstore Rekord Transparency Log.
Incentives	Vier-Augen-Prinzip via CODEOWNERS, automatisierte Dependabot-PRs reduzieren Reibung für sichere Updates, parallele Jobs + <code>cancel-in-progress</code> mildern den Geschwindigkeits-Konflikt (siehe Abschnitt 6.10).

Tabelle 1: Mapping der Pipeline-Bestandteile auf das Security-Engineering-Framework.

(ACL) oder zeilenweise als *Capability* realisiert. *Role-Based Access Control* (RBAC) bündelt Rechte über Rollen statt über einzelne Subjekte. Zwei Prinzipien sind hier zentral: das *Prinzip des geringsten Privilegs* und die *Mehr-Personen-Autorisierung* (Vier-Augen-Prinzip). In der untersuchten Pipeline finden sich diese Modelle direkt wieder: die per-Job vergebenen `permissions` sind capability-artige, minimal geschnittene Rechtebündel; GitHub-Environments und CODEOWNERS implementieren eine rollenbasierte Zugriffskontrolle auf Deployments bzw. auf sicherheitskritische Pfade (`/.github/`, `Dockerfile`); CODEOWNERS-Review und Production-Approval-Gate sind technische Umsetzungen der Mehr-Personen-Autorisierung.

Bell-LaPadula: Vertraulichkeit als Informationsfluss. Das *Bell-LaPadula*-Modell (BLP) [9, 8] formalisiert Vertraulichkeit als Informationsflusskontrolle über zwei Regeln: *No Read Up* (kein Lesen höher klassifizierter Daten) und *No Write Down* (kein Schreiben auf eine niedrigere Stufe). Übertragen auf die Pipeline ist ein Secret (Cloud-Credential, Signaturidentität) ein hoch klassifiziertes Objekt, auf das niedrig vertrauenswürdiger Code – etwa aus einem Fork-Pull-Request – nicht zugreifen darf. Der in Abschnitt 4.3 diskutierte `pull_request_target`-Angriff ist strukturell eine *No-Read-Up*-Verletzung. Die hier gewählte Trigger-Strategie (ausschließlich `pull_request`; Secret-Schritte per `if` gesperrt) stellt den Informationsfluss wieder her.

Biba und das Low-Water-Mark-Prinzip. Biba [10, 8] beobachtete, dass Integrität das *Dual* der Vertraulichkeit ist: Wo BLP regelt, *wer lesen darf*, regelt eine Integritätspolitik, *wer schreiben darf*. Das Biba-Modell kehrt die BLP-Regeln um (*No Read Down*, *No Write Up*), damit hochintegre Objekte nicht durch niedrigintegre, potenziell unzuverlässige Daten kontaminiert werden. Das zugehörige *Low-Water-Mark-Prinzip* besagt, dass die Integrität eines Objekts der niedrigsten Integritätsstufe aller an seiner Erzeugung beteiligten Objekte entspricht. Dieses Prinzip ist der theoretische Kern der Supply-Chain-Argumentation dieser Arbeit: Die Vertrauenswürdigkeit des fertigen Container-Images ist durch die am wenigsten vertrauenswürdige Komponente begrenzt, die in seinen Build ein-

geflossen ist – eine transitive Abhängigkeit, ein Base-Image oder ein Build-Schritt. SBOM und SLSA-Provenance (Abschnitte 5.5 und 6.1) machen diese Low-Water-Mark explizit, prüfbar und signiert; Dependency-Pinning, `pip-audit` und Trivy verhindern, dass niedrigintegre Inputs unbemerkt „nach oben“ fließen. Die Pipeline realisiert damit faktisch eine mehrstufige Integritätspolitik im Sinne Bibas, auch ohne explizite Integritätslabel.

Authentifizierungsprotokolle: Frische und Replay. In der protokollanalytischen Sicht der Lehrveranstaltung gilt *Authentizität = Integrität + Aktualität (freshness)*: Ein Authentifizierungsprotokoll muss neben der Identität auch die *Frische* einer Nachricht garantieren, um Replay-Angriffe auszuschließen. Die OIDC-Federation der Pipeline ist genau dies – die ausgestellten Tokens sind kurzlebig und an die Workflow-Run-ID gebunden, sodass ein abgefangenes Token nach Minuten wertlos ist. Cosign-Keyless-Signing und das Rekord-Transparency-Log etablieren analog eine überprüfbare Vertrauensbeziehung ohne langlebiges Schlüsselmaterial.

3. System unter Betrachtung

3.1. Architekturüberblick

Untersucht wird eine prototypische, container-basierte Pipeline für eine Python-Webanwendung. Die wesentlichen Stufen sind in Abbildung 1 dargestellt.

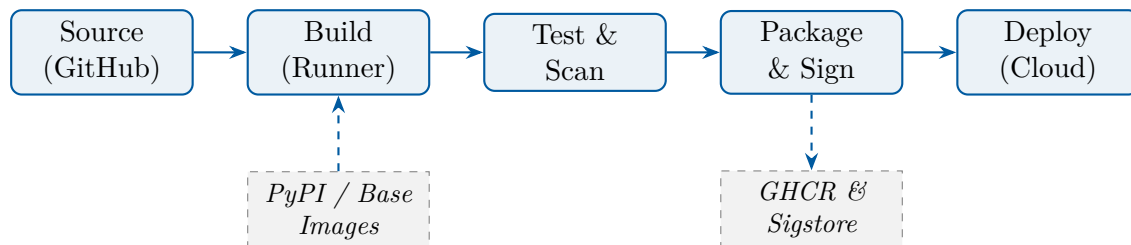


Abbildung 1: Pipeline-Stufen und externe Vertrauensanker.

3.2. Schützenswerte Assets

Asset	Schutzziel (CIA + AAA)
Quellcode (main)	Integrität, Authentizität
Build-Artefakt (Image)	Integrität, Authentizität, Verfügbarkeit
Secrets (Cloud-Cred., Tokens)	Vertraulichkeit, Integrität
SBOM & Provenance	Integrität, Nicht-Abstreitbarkeit
Pipeline-Konfiguration	Integrität, Autorisierung
Build-Runner-Identität (OIDC)	Authentizität, Vertraulichkeit

3.3. Trust Boundaries

1. Entwickler-Workstation ↔ GitHub
2. GitHub-Plattform ↔ Build-Runner
3. Build-Runner ↔ Paket-/Image-Registries (PyPI, Docker Hub, GHCR)
4. Build-Runner ↔ Sigstore (Fulcio/Rekor)
5. Pipeline ↔ Deployment-Target (Cloud-API)

3.4. Angreifermodell

Die Lehrveranstaltung kategorisiert relevante Gegnertypen entlang ihrer Motivation, Ressourcen und Methoden: Cyberkriminelle, staatliche Akteure (Nachrichtendienste), unehrliche Insider, konkurrierende Unternehmen sowie – im Sinne Andersons – aktivistisch motivierte Gruppen („wütender Mob“). Diese Kategorien werden für die vorliegende Arbeit auf drei DevSecOps-relevante Profile verdichtet, die entlang der Trust Boundaries (Abschnitt 3.3) operieren:

A1 – Externer Angreifer

kein Repo-Zugriff; nutzt Phishing, Typosquatting, Schwachstellen in Abhängigkeiten und Tag-Hijacks in öffentlichen Registries. Deckt die Vorlesungs-Kategorien *Cyberkriminelle* (finanziell motiviert: Crypto-Mining auf Runnern, Ransomware) und *Hacktivismus* (z. B. Protestware in npm-Paketen) ab.

A2 – Kompromittierter Mitwirkender

gültige, aber begrenzte Schreibrechte – Fork-PR-Autor, einzelner Maintainer mit übernommenem Account, oder ein Maintainer einer als Dependency eingezogenen Bibliothek. Entspricht der Vorlesungskategorie *unehrliche Insider* und ist zugleich der typische Eintrittsvektor für staatliche Supply-Chain-Operationen (SolarWinds-Stil, vgl. Walkthrough 1 in Abschnitt 4.2).

A3 – Insider/Plattform

Admin-Rechte am Repo, kompromittierter Self-hosted-Runner, oder ein staatlicher Akteur mit Zugriff auf einen Vertrauensanker (Registry, Sigstore, GitHub). Vereint die Vorlesungs-Kategorien *Nachrichtendienste* (ressourcenstark, verdeckt) und *Insider mit Admin-Rechten*; in beiden Fällen ist der Angreifer innerhalb des deklarierten Vertrauensbereichs der Pipeline.

Die Differenzierung in drei Profile ist bewusst schmal gehalten: Eine feinere Aufschlüsselung würde die folgende STRIDE-Analyse (Tab. 3) künstlich aufblähen, ohne neue Mitigationsentscheidungen sichtbar zu machen. Die Vorlesungs-Kategorie *konkurrierende Unternehmen* wird hier nicht eigenständig modelliert, da deren Angriffsvektoren technisch deckungsgleich mit A1 oder A2 sind und sich lediglich in der Motivation unterscheiden.

4. Bedrohungsanalyse (STRIDE)

4.1. STRIDE-Mapping pro Pipeline-Stufe

Die folgende Tabelle dokumentiert pro Pipeline-Stufe die wesentlichen Bedrohungen, klassifiziert nach STRIDE. Die Risiko-Einstufung folgt einer qualitativen Skala (niedrig/mittel/hoch) auf Basis von Eintrittswahrscheinlichkeit und Schadensausmaß im Kontext einer Studienarbeitstypischen Demo-Pipeline.

Stufe	STRIDE	Bedrohung	Angr.	Risiko
Source	S	Push mit gestohlenen Maintainer-Credentials	A1/A2	hoch
Source	T	Manipulation von Pipeline-Definitionen via PR	A2	hoch
Source	R	Unsignierte Commits, fehlende Audit-Trails	A2	mittel
Source	I	Secrets im Klartext im Repo (Logs, Configs)	A1	hoch
Build	T	Bösartige transitive Dependency (Confused-Deps, Typosquat)	A1	hoch
Build	T	Manipulation des Base-Images (Tag-Replacement)	A1	hoch
Build	E	Workflow mit zu weiten permissions (RW everywhere)	A2	hoch
Build	E	<code>pull_request_target</code> mit <code>secrets</code> ausgesetzt	A1	hoch
Build	I	Cache-Poisoning zwischen PRs	A2	mittel
Test	D	Lange laufende Scans blockieren Deployments	A1	niedrig
Test	T	Test-Bypass via geänderte Workflow-Bedingungen	A2	mittel
Pkg	T	Ungesignedes Image, Tag-Hijack in Registry	A1	hoch
Pkg	R	Fehlende Provenance / SBOM	–	mittel
Deploy	S	Long-lived Cloud-Credentials in Repo-Secrets	A2	hoch
Deploy	E	Fehlende Approval-Gates für Production	A2	hoch
Deploy	I	Verbose Logs leaken Tokens / PII	A1	mittel

Tabelle 3: STRIDE-Mapping der untersuchten Pipeline.

4.2. Threat Walkthroughs

Die kondensierte Tabellenform aus Tabelle 3 verdichtet Bedrohungen, verschleiert aber ihre tatsächliche *Verkettung*. Reale Angriffe auf CI/CD-Systeme bestehen selten aus einem einzelnen Schritt; charakteristisch ist die schrittweise Eskalation entlang der Trust

Boundaries. Im Folgenden werden drei repräsentative Bedrohungen aus Tabelle 3 als ausformulierte Angriffsketten dargestellt – inklusive Voraussetzungen, Eskalationsschritten, Erkennungs- und Mitigationenpunkten.

Walkthrough 1 – Bösartige transitive Dependency (Build-T, A1). *Voraussetzung:* Eine populäre, indirekt eingezogene Abhängigkeit wird durch einen kompromittierten Maintainer-Account aktualisiert; die neue Version enthält eine Post-Install-Routine, die Umgebungsvariablen und `~/.aws/credentials` an einen externen HTTP-Endpunkt sendet.

Schritt 1 – Aufnahme. Im Zielprojekt erstellt Dependabot plangemäß einen wöchentlichen Update-PR. Reviewer prüfen den Diff der `requirements.txt`, sehen jedoch nur einen scheinbar harmlosen Versionssprung der direkten Abhängigkeit; die *transitive* Aktualisierung ist im Diff nicht sichtbar.

Schritt 2 – Build-Stufe. Der CI-Runner installiert die Abhängigkeit; die Post-Install-Routine läuft mit den Umgebungs-Permissions des Workflows.

Schritt 3 – Exfiltration. Der Schadcode greift auf `$GITHUB_TOKEN` und auf etwaig gemountete Cloud-Credentials zu und sendet sie an den Angreifer-Endpunkt.

Erkennung. `pip-audit` prüft die finale Resolverliste gegen die OSV-Datenbank, fängt aber nur Versionen mit bereits öffentlich bekannter CVE. Eine *frische* Kompromittierung wird in den ersten Stunden nicht erkannt – hier wäre eine zweite Verteidigungslinie nötig: Der als Erweiterung vorgesehene SBOM-Diff (Abschnitt 5.5) würde das neue Paket sichtbar hervorheben, und Egress-Restriktionen auf dem Runner können den Exfiltrationsversuch blockieren.

Mitigation in der Pipeline. Drei Maßnahmen wirken zusammen: (1) `permissions: {}` als Default reduziert die Reichweite des `GITHUB_TOKEN`; (2) `persist-credentials: false` bei `actions/checkout` verhindert, dass das Token im `.git/config` liegen bleibt; (3) die Cloud-Federation per OIDC ohne langlebige Secrets im Repo entzieht dem Angreifer das wertvollste Ziel.

Restrisiko. Der Angriff bleibt möglich, solange transitive Updates nicht zwingend an einer kuratierten Allowlist hängen; eine weitergehende Härtung wäre der Einsatz eines Privat-Mirrors mit Repository-Manager (z. B. Sonatype Nexus) und manueller Freigabe neuer Versionen.

Walkthrough 2 – Manipulation der Pipeline-Definition via PR (Source-T, A2). *Voraussetzung:* Ein Kontributor mit „Triage“-Rolle (oder ein Forker mit angenommenem PR-Template) reicht einen unscheinbaren Pull-Request ein, der nicht den Anwendungscode, sondern eine Datei unter `.github/workflows/` ändert – etwa um einen zusätzlichen Schritt einzufügen, der `secrets.AWS_DEPLOY_KEY` in einen publizistischen Webhook schreibt.

Schritt 1 – Tarnung. Der PR ist optisch ein „Refactoring“: Der Workflow wird in mehrere Teildateien zerlegt, dabei werden zusätzliche `run:-`Schritte eingefügt. Reviewer ohne explizit gepflegte CODEOWNERS-Regeln neigen dazu, den Diff zu überfliegen.

Schritt 2 – Trigger. Der Angreifer wartet darauf, dass ein Maintainer den PR mergt. Sobald der Workflow auf `main` läuft, führt der eingefügte Schritt aus – mit den vollen `secrets.~`-Werten der Pipeline.

Erkennung. CODEOWNERS auf `/.github/` verschiebt den Review zwingend an die im Repo deklarierten Pipeline-Verantwortlichen; OpenSSF Scorecard markiert das Fehlen dieser Regel als Posture-Defizit. `actionlint` und `zizmor` können in der CI selbst auf typische PPE-Antipatterns prüfen.

Mitigation in der Pipeline. CODEOWNERS auf `/.github/`, Required-Status-Checks für jede Pipeline-Änderung, sowie die Regel, dass `secrets` nur in Jobs verfügbar sind, die einem Environment mit Approval zugeordnet sind.

Restrisiko. Ein Insider mit Admin-Rechten kann CODEOWNERS selbst ändern. Hier hilft nur Org-weites Audit-Logging mit Alarm bei Änderungen an Branch-Protection-Konfigurationen.

Walkthrough 3 – Tag-Hijack in der Registry (Pkg-T, A1). *Voraussetzung:* Der Angreifer hat sich Schreibzugriff auf den Registry-Namespace verschafft – z.B. über einen kompromittierten Service-Account-Token, der versehentlich in einem öffentlichen Issue gepostet wurde.

Schritt 1 – Repush. Der Angreifer baut lokal ein böses Image, vergibt denselben Tag wie das letzte Release (`ghcr.io/org/app:1.4.2`) und pusht es. Der Manifest-Digest ändert sich; der Tag zeigt jetzt auf das neue Manifest.

Schritt 2 – Pull bei Deployment. Ein Deployment-Skript, das nur den Tag referenziert, zieht beim nächsten Rollout das manipulierte Image.

Erkennung. Der CD-Workflow verifiziert vor dem Rollout per `cosign verify` sowohl die Signatur (gegen die OIDC-Identität des CI-Workflows) als auch die SLSA-Provenance-Attestation. Beide Checks schlagen für das manipulierte Image fehl, da der Angreifer keine gültige Signatur unter dem CI-Workflow-OIDC-Issuer erzeugen kann; das Rekord-Transparency-Log macht solche Manipulationen darüber hinaus öffentlich auditierbar.

Mitigation in der Pipeline. Im CD-Workflow erfolgt die Verifikation vor jedem Deploy; die Deploy-Schritte referenzieren ausschließlich den Image-Digest (`@sha256:...`), nicht den Tag.

Restrisiko. Ein Angriff auf Sigstore selbst ist nicht unmittelbar mitigierbar – daher die Listung von Sigstore als Vertrauensanker in Abschnitt 3.

4.3. Vertiefung: Poisoned Pipeline Execution (PPE)

Die Bedrohungen *Build-T* (Manipulation Pipeline-Definition) und *Build-E* (`pull_request_target` mit Secrets) aus Tabelle 3 sind manifestationen einer übergeordneten Klasse, die OWASP als *Poisoned Pipeline Execution* [6] beschreibt. PPE bezeichnet jeden Angriff, der den „Konfigurations-als-Code“-Charakter moderner CI-Systeme ausnutzt: Wer den Workflow ändern darf, kontrolliert – transitiv – alle Aktionen, die in seinem Kontext ablaufen. Aus Sicht der Zugriffskontrollmodelle (Abschnitt 2.5) ist PPE eine kombinierte Verletzung

zweier dualer Eigenschaften: Niedrigintegrity Code erlangt Schreibwirkung im hochintegren Pipeline-Kontext (Biba, *No Write Up*) und zugleich Lesezugriff auf hoch klassifizierte Secrets (BLP, *No Read Up*); eine wirksame Mitigation muss daher beide Flussrichtungen schließen.

Direkte und indirekte PPE. Direkte PPE liegt vor, wenn der Angreifer die Pipeline-Definition selbst modifiziert (z. B. als Maintainer mit Push-Recht). Indirekte PPE nutzt Trigger aus, die *trotz* fehlender Schreibrechte Code im Kontext der Pipeline ausführen. Der prominenteste Vektor in GitHub Actions ist der Trigger `pull_request_target`.

Anatomie von `pull_request_target`. Im Gegensatz zum Standard-Trigger `pull_request` – der den Workflow im Kontext des PR-Branche ohne Zugriff auf `secrets` ausführt – läuft `pull_request_target` im Kontext des Ziel-Branche *mit* vollem Secret-Zugriff. Der Trigger wurde geschaffen, um Aktionen wie automatisches Labeling auch für Forks zu ermöglichen, bei denen der Forker keinen Schreibzugriff auf das Ziel-Repository besitzt. Genau diese Kombination (Code-Quelle = Fork, Secrets = Original-Repo) ist die Kernschwäche.

Ein typischer fehlerhafter Workflow checkt zusätzlich den PR-HEAD aus und führt dort Skripte aus:

```
1 on:
2   pull_request_target:           # laeuft im Ziel-Repo-Kontext mit Secrets
3     types: [opened, synchronize]
4 jobs:
5   ci:
6     steps:
7     - uses: actions/checkout@<sha>    # OK: HEAD = Ziel-Branch
8       with:
9         ref: ${github.event.pull_request.head.sha} # PROBLEM!
10    - run: npm ci && npm test          # PROBLEM: laedt + fuehrt Fork-
      Code aus
```

Listing 1: Anti-Pattern: `pull_request_target` mit unsicherem Checkout

Ein bössartiger Forker fügt seinem PR-Branch einen `postinstall`-Hook in der `package.json` hinzu, der die Werte aller Umgebungsvariablen exfiltriert. Beim ersten `npm ci` im Workflow läuft dieser Hook – inklusive Zugriff auf `secrets.NPM_TOKEN`, `secrets.AWS_*` und das `GITHUB_TOKEN` mit Schreibrecht auf das Original-Repo.

Codecov 2021 als Realbeispiel. Im April 2021 wurde der Codecov-Bash-Uploader durch einen Fehler in einem Docker-Image-Build kompromittiert; in der Folge konnten Angreifer rund zwei Monate lang Umgebungsvariablen aus CI-Umgebungen exfiltrieren, die den Uploader integriert hatten. Der Vorfall war zwar kein klassischer PPE-Fall, illustriert aber die gleiche Kausalkette: ein in einem Pipeline-Schritt ausgeführtes Skript, das implizit Zugriff auf alle Secrets erhält, die im Kontext des Workflows verfügbar sind. Branchenweite Reaktion war eine Auseinandersetzung mit dem Trigger-Modell und der Secret-Sichtbarkeit.

Mitigation in der hier umgesetzten Pipeline. Die folgenden konstruktiven Entscheidungen schließen die PPE-Klasse weitgehend:

- **Kein `pull_request_target`.** Der CI-Workflow verwendet ausschließlich den sicheren `pull_request`-Trigger; Aktionen mit Secret-Bedarf (z. B. Image-Push) werden bei PRs per `if: github.event_name != 'pull_request'` ausgesperrt.
- **CODEOWNERS auf `/.github/`.** Jede Workflow-Änderung erfordert ein Review der designierten Pipeline-Verantwortlichen; das Vier-Augen-Prinzip wird damit auf die kritischste Datei-Klasse angewendet.
- **Default-deny permissions: `{}`.** Selbst wenn ein PPE-Angriff Code im Workflow-Kontext zur Ausführung bringt, ist das verfügbare `GITHUB_TOKEN` ohne Schreibrechte und kann weder Branches noch Releases manipulieren.
- **OIDC-Federation statt langlebiger Secrets.** Cloud-Zugriffe verwenden kurzlebige, an die OIDC-Identität des Workflows gebundene Tokens. Selbst eine erfolgreiche Exfiltration nutzt dem Angreifer wenig, da die Tokens auf Minutenbasis ablaufen und an die Workflow-Run-ID gebunden sind.
- **persist-credentials: false.** Der `GITHUB_TOKEN` wird nach `actions/checkout` nicht in `.git/config` verewigt; ein nachgelagertes Skript hat damit keinen Hebel mehr.

Verbleibende PPE-Risiken. Eine vollständige Immunität ist nicht erreichbar. Eine Kompromittierung der GitHub-Plattform selbst, eine Schwachstelle in einer als Dependency genutzten Action (vgl. Fallstudie `tj-actions` in Abschnitt 5.9) oder ein Insider mit Administrationsrechten am Repo bleiben außerhalb der Reichweite der hier diskutierten Maßnahmen. Diese Risiken werden in Abschnitt 6.8 explizit als Restrisiken geführt.

5. Gegenmaßnahmen und Implementierung

Die folgenden Maßnahmen lassen sich als konkrete Instanzen der in Abschnitt 2.5 eingeführten Modelle lesen: least-privilege `permissions`, CODEOWNERS und Environments realisieren Zugriffskontrolle und Mehr-Personen-Autorisierung, während Pinning, SBOM und Provenance die Biba-Integritätspolitik (Low-Water-Mark) durchsetzen.

5.1. Source-Stufe

Branch Protection & CODEOWNERS

`main` ist geschützt; Merges erfordern ein Review durch Code-Owner (Vier-Augen-Prinzip), bestandene Pflicht-Checks, signierte Commits und lineare History.

Signierte Commits

Entwickler signieren mit SSH/GPG; nicht-signierte Commits werden serverseitig abgelehnt.

Secret-Scanning

`gitleaks` läuft als Pre-Commit-Hook und in der CI; GitHub Push-Protection ergänzt das.

Pre-Commit-Hooks

`.pre-commit-config.yaml` spiegelt die zentralen CI-Checks lokal: `ruff` (Lint), `bandit` (`-ll -ii`, identische Schwellwerte wie die CI) und `gitleaks`. Entwickler sehen Findings damit vor dem Push – derselbe Schutz wird zur schnellsten Option (vgl. Abschnitt 6.10).

Pinned Actions

Alle GitHub-Actions sind auf Commit-SHA gepinnt, um Tag-Retag-Angriffe (z. B. `tj-actions/changed-files 03/2025`) zu verhindern.

Vulnerability-Disclosure

`SECURITY.md` definiert den koordinierten Disclosure-Prozess (5-Werkstage-Reaktionszeit, 14-Tage-Fix für High-Severity, GitHub Private Vulnerability Reporting); flankiert die CRA- und SSDF-RV-Anforderungen aus Abschnitt 6.7.

5.2. Build-Stufe

Least-Privilege-Permissions

Workflow-Default ist `permissions: {}`; jeder Job opt-in mit minimalen Scopes.

Ephemere Runner

Nutzung von GitHub-hosted Runnern (frische VM pro Job); Self-hosted bewusst nicht verwendet.

Reproduzierbares Image

Multi-stage Dockerfile mit gepinntem Base-Image, Non-Root-User (UID 10001), Read-only-Filesystem-tauglich.

Dependency-Pinning

`requirements.txt` mit exakten Versionen; Updates über Dependabot-PRs, die die volle Pipeline durchlaufen.

5.3. Test- und Scan-Stufe

- **SAST:** Bandit (Python) + GitHub CodeQL, Ergebnisse als SARIF in die GitHub Code-Scanning-API.
- **SCA:** `pip-audit` gegen die OSV-Datenbank, `-strict` bricht den Build bei bekannter CVE ab.
- **Container-Scan:** Trivy auf das gebaute Image, `HIGH,CRITICAL` mit `exit-code: 1`.
- **Coverage-Gate:** `pytest -cov-fail-under=80`.

5.4. Package- und Sign-Stufe

SBOM

Syft erzeugt CycloneDX-JSON aus dem Image; das SBOM wird als Attestation an das Image angeheftet.

Cosign Keyless Signing

Signatur via Sigstore Fulcio mit dem OIDC-Token des Workflows; keine langlebigen Schlüssel im Repo.

SLSA-Provenance

`actions/attest-build-provenance` erzeugt eine verifizierbare Provenance-Attestation (SLSA v1).

5.5. SBOM-Lebenszyklus

Eine *Software Bill of Materials* (SBOM) listet alle direkten und transitiven Komponenten eines Artefakts auf. Sie ist kein einzelner Sicherheitsmechanismus, sondern der zentrale *Datensatz*, auf dem mehrere Mechanismen aufsetzen: Schwachstellen-Diff, Lizenz-Tracking, Compliance-Nachweise und langfristige Reproduzierbarkeit. Die Implementierung folgt dem in Abbildung 2 skizzierten Lebenszyklus.

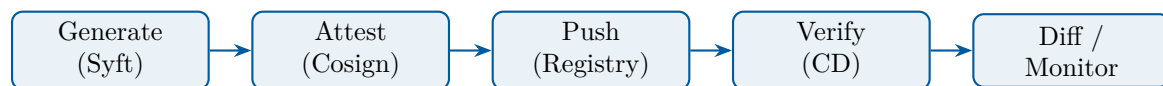


Abbildung 2: SBOM-Lebenszyklus: Erzeugung, Attestation, Verteilung, Verifikation, kontinuierliches Monitoring.

1. Erzeugung mit Syft. Syft (Anchore) inspiziert das gebaute Image schichtweise und erkennt Pakete von Distribution-Manager (`apt`), Sprach-Manager (`pip`, `npm`, `gem`) sowie statisch eingebundene Bibliotheken. Ausgabeformat ist CycloneDX in JSON, weil dies zusätzlich zur reinen Komponentenliste auch *Beziehungen* (`depends-on`, `contains`) und Hashes erfasst – beides ist für spätere Verifikation entscheidend.

2. Attestation der SBOM. Statt das SBOM lose neben dem Image zu lagern (klassisches `sbom.json` im Release-Asset-Ordner), wird es mit `actions/attest-sbom` als signierte, keyless über die OIDC-Identität des Workflows erzeugte *Attestation* an den Image-Digest gebunden und in die Registry geschoben. Damit ist das SBOM kryptographisch an genau das Image gekettet, das es beschreibt – und manipulierte SBOM-Kopien lassen sich erkennen.

3. Verteilung über die Registry. Cosign legt Attestations als zusätzliche OCI-Manifeste neben dem eigentlichen Image ab; Pull-Clients holen sie auf Wunsch (`cosign download attestation`). Vorteile: keine zweite Distributionspipeline, keine zusätzlichen Zugriffsrechte, keine abweichende Aufbewahrungsdauer.

4. Verifikation vor Deploy. Der CD-Workflow verifiziert vor dem Rollout die Image-Signatur (`cosign verify`) sowie die *SLSA-Provenance-Attestation* (`cosign verify-attestation -type slsaprovenance`), jeweils gegen die erwartete CI-Workflow-Identität (vgl. Listing 3). Die SBOM-Attestation ist an denselben Image-Digest gebunden und lässt sich unabhängig prüfen (`gh attestation verify` bzw. `cosign`), ist im aktuellen Prototyp

jedoch nicht Teil des verpflichtenden Deploy-Gates. Eine Manipulation an Signatur oder Provenance wird damit *vor* dem Rollout sichtbar.

5. Kontinuierliches Monitoring (SBOM-Diff). Der eigentliche operative Wert eines SBOM entsteht erst nach Auslieferung. Der hier skizzierte SBOM-Diff ist im aktuellen Prototyp *nicht* umgesetzt, sondern als Erweiterung vorgesehen (vgl. Kapitel 7): Bei jedem Build würde ein nachgelagerter Schritt die neue SBOM gegen die letzte gepushte Version desselben Stream-Tags vergleichen. Drei Fragen werden dabei beantwortet:

1. **Welche Pakete sind hinzugekommen?** Neue transitive Abhängigkeiten sind das primäre Eintrittsfenster für Supply-Chain-Angriffe (vgl. Walkthrough 1 in Abschnitt 4.2).
2. **Welche Versionen haben sich geändert?** Ein Versionssprung über mehrere Major-Releases hinweg ist ein Signal für übersehene Major-Updates und sollte einen Reviewer zwingen, das Changelog zu prüfen.
3. **Sind bekannte CVEs hinzugekommen?** Werkzeuge wie `grype` oder `trivy sbom` korrelieren die SBOM gegen die OSV/NVD-Datenbanken; das ist günstiger als ein kompletter Image-Scan und lässt sich gegen *historische* Artefakte ausführen, die in Produktion noch laufen.

6. Aufbewahrung und Auskunft. SBOMs müssen über die Lebensdauer des Artefakts verfügbar bleiben – für eine Library, die sieben Jahre verkauft wird, also potenziell deutlich länger als ein typischer Container-Lebenszyklus (vgl. die in Abschnitt 2 erwähnte Software-Sustainability-Diskussion). Die Bindung der Attestation an den Image-Digest und die Hinterlegung im Sigstore-Rekor-Log adressiert das auch dann, wenn die ursprüngliche Pipeline-Konfiguration nicht mehr existiert.

Bezug zum Cyber Resilience Act (CRA). Mit dem *Cyber Resilience Act* [7] der EU werden SBOMs ab Ende 2027 für „Produkte mit digitalen Elementen“ eine verpflichtende technische Dokumentation. Hersteller müssen eine SBOM für jedes vermarktete Produkt führen und Behörden auf Anfrage verfügbar machen; bei „important“ bzw. „critical“ Produkten kommen erweiterte Pflichten hinzu (Conformity Assessment, Vulnerability Handling, Coordinated Disclosure). Die hier umgesetzte Pipeline erfüllt die kommenden technischen Anforderungen *by construction*: SBOM wird automatisiert erzeugt, an das Artefakt gebunden, signiert verteilt und ist über das Rekor-Log nachvollziehbar. Die organisatorischen CRA-Pflichten (z. B. 24h-Reporting bei aktiv ausgenutzten Schwachstellen) bleiben außerhalb des Pipeline-Scopes, sind aber durch die SECURITY.md-Policy und das Vulnerability-Disclosure-Verfahren des Repositories vorbereitet.

Grenzen. Eine SBOM ist nur so vollständig wie der Generator. Statisch eingebettete C-Bibliotheken in Wheel-Distributionen, vendoring-basierte Build-Systeme und nachgeladene Plugins entgehen einer rein paketmanager-basierten Erkennung; entsprechende Pakete landen unentdeckt im Image. Die hier verwendete Kombination aus Multi-Stage-Dockerfile mit gepinnten Versionen und Container-Scan (Trivy) auf das fertige Image kompensiert das teilweise, beseitigt aber nicht das grundsätzliche Vollständigkeitsproblem.

5.6. Deploy-Stufe

Die Deploy-Jobs des CD-Workflows sind als illustrative Platzhalter ausgeführt – der eigentliche Cloud-Login und das Rollout via `kubectl` liegen gemäß Abschnitt 1.3 außerhalb des Scopes; die sicherheitsrelevanten Invarianten (Verifikation und Digest-Pinning) sind hingegen real umgesetzt.

OIDC-Federation

Die Deploy-Jobs deklarieren `id-token: write` für eine Cloud-Federation (Azure / AWS) via OIDC; statische Cloud-Credentials in Repo-Secrets werden bewusst vermieden.

Environments mit Approval

`staging` und `production` sind GitHub-Environments; Pflicht-Reviewer und Wait-Timer für `production` werden in den Environment-Einstellungen konfiguriert (vgl. README).

Cosign-Verify vor Deploy

Vor dem Rollout verifiziert der CD-Workflow Signatur *und* SLSA-Provenance des Image-Digests.

Image-Pinning per Digest

Deployments referenzieren `@sha256:...`, nicht Tags.

5.7. Auszug aus der CI-Implementierung

Listing 2 zeigt die Job-Topologie und die zentralen Sicherheits-Gates; die vollständigen Workflow-Dateien sowie Dockerfile, CODEOWNERS und die Dependabot-Konfiguration sind in Anhang A dokumentiert.

```
1 permissions: {}                                # default-deny
2
3 jobs:
4   lint-test:
5     permissions: { contents: read }
6     steps:
7       - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
8         # v4.2.2
9       with: { persist-credentials: false }
10  build-image:
11    needs: [lint-test, sast, sca, secret-scan]
12    permissions:
13      contents: read
14      packages: write
15      id-token: write          # OIDC fuer cosign keyless
16      attestations: write
```

Listing 2: Auszug `.github/workflows/ci.yml`: Permissions & Pin auf SHA

```
1 cosign verify \
2   --certificate-identity-regexp "https://github.com/$REPO/.github/
3   workflows/.+" \
4   --certificate-oidc-issuer "https://token.actions.githubusercontent.com
5   " \
```

```

4  "$IMAGE"
5
6  cosign verify-attestation --type slsaprovenance \
7    --certificate-identity-regexp "https://github.com/$REPO/.github/
    workflows/." \
8    --certificate-oidc-issuer "https://token.actions.githubusercontent.com
    " \
9  "$IMAGE"

```

Listing 3: Verifikation vor Deploy (cd.yml)

5.8. Mapping Bedrohung → Maßnahme

Bedrohung (Tab. 3)		Maßnahme
Maintainer-Credential-Diebstahl		MFA, signierte Commits, CODEOWNERS-Review
Manipulation	Pipeline-Definition	CODEOWNERS auf .github/, Required Checks
Bösartige Dependency		pip-audit, Dependabot, Pinning, SBOM-Attestation
Base-Image-Replacement		Pinning + Trivy + Image-Digest in Deploy
Zu weite permissions		default-deny + per-Job opt-in
pull_request_target Leak		Trigger nicht verwendet; PRs ohne Secrets
Tag-Hijack in Registry		Cosign-Verify + Digest-Pinning beim Deploy
Fehlende Provenance		SLSA-Provenance via attest-build-provenance
Long-lived Cloud-Creds		OIDC-Federation, keine statischen Secrets
Fehlendes Approval	Pro-duction	GitHub Environment Reviewer + Wait-Timer

Tabelle 4: Bedrohungs-Maßnahmen-Matrix.

5.9. Fallstudie: tj-actions/changed-files (März 2025)

Die Action `tj-actions/changed-files` wurde im März 2025 kompromittiert: Angreifer überschrieben mehrere veröffentlichte Tags so, dass sie auf einen manipulierten Commit zeigten, der Secrets aus dem Runner-Memory in die Build-Logs exfiltrierte. Workflows, die die Action ohne SHA-Pinning eingebunden hatten (`tj-actions/changed-files@v44`), zogen automatisch die manipulierte Version. Über zehntausend Repositories waren betroffen, in mehreren Fällen wurden GitHub-PATs und Cloud-Credentials öffentlich auf GitHub-Logs sichtbar.

Der Vorfall illustriert exakt die Bedrohungsklasse *Source/Build-T* aus Tabelle 3. Die in dieser Arbeit umgesetzte Maßnahme – durchgängiges Pinning aller Actions auf den Commit-SHA mit einem nachgestellten Versions-Kommentar – hätte den Angriff vollständig neutralisiert: Selbst wenn der Tag `v44` auf einen bösartigen Commit umgehängt wird, verweist der Pin `tj-actions/changed-files@<sha> # v44` weiterhin auf die auditierte Version. Sigstore/Cosign hätte die Auswirkung zusätzlich begrenzt, da das resultierende Artefakt keine gültige Signatur unter der OIDC-Identität des Workflows getragen hätte.

6. Evaluation

6.1. SLSA-Einstufung

Supply-chain Levels for Software Artifacts (SLSA) definiert in Version 1.0 [1] eine ordinale Skala für die Vertrauenswürdigkeit der Build-Plattform und des Quellcode-Prozesses. Die Skala besteht aus zwei orthogonalen Tracks (*Build* und *Source*); die jeweiligen Levels sind kumulativ, d. h. jedes höhere Level fordert die Anforderungen aller niedrigeren mit. Im Folgenden werden die Levels zusammengefasst, mit der konkreten Umsetzung im untersuchten System abgeglichen und die Begründung für *nicht* erreichte höhere Stufen transparent gemacht.

Build-Track.

L0

Keine besonderen Anforderungen. Der Build kann lokal, manuell, ohne jede Provenance erfolgen.

L1

Provenance existiert in irgendeiner Form – der Build ist also reproduzierbar dokumentiert. Inhalt und Authentizität der Provenance werden noch nicht abgesichert.

L2

Provenance ist *signiert* und wird von einem *gehosteten Build-Service* erzeugt. Der Build läuft also nicht mehr auf einer Entwickler-Workstation; der Build-Service kann Provenance an seine eigene Identität binden.

L3

Der Build-Service garantiert zusätzlich *Non-Forgeable Provenance* und *Isolation* zwischen Builds: Provenance kann nicht durch den Build-Schritt selbst manipuliert werden, und ein Build kann nicht mit Artefakten oder Geheimnissen vorheriger Builds interagieren.

Die hier umgesetzte Pipeline erreicht **Build L3**. Konkret:

- GitHub-hosted Runner (ubuntu-24.04) ist ein gehosteter Service; jeder Job läuft in einer frisch provisionierten VM, was die Isolationsanforderung erfüllt (*ephemeral builder*).
- Provenance wird mit `actions/attest-build-provenance` erzeugt und über die OIDC-Identität des Workflows signiert; das Signaturmaterial liegt nie im für die Build-Schritte zugreifbaren Speicher (Non-Forgeable).
- Das Build-Skript (`Dockerfile` und Workflow) ist versioniert, mit Branch-Protection und CODEOWNERS-Review geschützt.

Source-Track.

L1

Quellcode ist versioniert (z. B. Git); Änderungen sind rückverfolgbar.

L2

Zusätzlich *verifizierte Geschichte*: Commits sind kryptographisch signiert; der Repository-Provider verifiziert Identitäten beim Push.

L3

Zusätzlich *Two-person review* und Schutz vor Force-Push / History-Rewrite.

Die Pipeline erreicht **Source L3**: **main** ist branch-protected mit CODEOWNERS-Review (Vier-Augen), Force-Push und History-Rewrite sind serverseitig deaktiviert, signierte Commits sind Pflicht-Status-Check.

Warum nicht höher? SLSA v1.0 definiert offiziell nur Build-L0 bis L3 und Source-L1 bis L3; es gibt also kein „L4“, das hier zu erreichen wäre. Konzeptionell relevant für eine Studienarbeit ist dennoch die Diskussion, welche *darüber hinausgehenden* Eigenschaften ein „strenger als L3“-Setup hätte:

1. **Hermetische Builds** – der Build hat keinen Netzwerkzugriff zur Build-Zeit; alle Inputs sind vorab in einem Lockfile mit Hash-Pinning erfasst. Die hier verwendeten Standard-pip-Installs verletzen diese Eigenschaft, da der Resolver Indizes anfragt. Eine vollständige Härtung würde `pip install --require-hashes` mit erzeugten `hashin`-Lockfiles und einem Egress-Restricted Runner erfordern.
2. **Reproduzierbare Builds** – bit-für-bit identische Artefakte aus identischen Inputs. Container-Images sind hier notorisch problematisch (Build-Zeitstempel, Layer-IDs, Mtime in Tar-Headern). Werkzeuge wie `kaniko` mit `--reproducible` oder `buildkit`'s `SOURCE_DATE_EPOCH` adressieren das, sind aber im Standard-docker/build-push-action-Setup nicht aktiviert.
3. **Multi-Party-Builds** – zwei unabhängige Build-Plattformen produzieren dasselbe Artefakt; Abweichungen sind ein Alarmsignal. Das ist außerhalb der Reichweite einer Pipeline auf einem einzigen Provider.

Diese Eigenschaften werden im Ausblick (Kapitel 7) als weiterführende Forschungsrichtungen benannt. Für den Studienarbeit-Scope ist die kombinierte Erreichung von Build-L3 + Source-L3 ein bewusst gesetzter Höchststand, der mit handelsüblichen Mitteln (GitHub Actions + Sigstore) ohne kommerzielle Add-ons darstellbar ist.

Verifikation der Einstufung. Die Selbst-Einstufung wird durch externe Werkzeuge geengeprüft: OpenSSF Scorecard (Abschnitt 6.2) bewertet einzelne SLSA-relevante Checks objektiv; `actions/attest-build-provenance` produziert maschinenlesbare Provenance, die mit `slsa-verifier` gegen die deklarierten Anforderungen geprüft werden kann.

Einordnung als Reifegradmodell. SLSA ist im Sinne der Lehrveranstaltung ein *Reifegradmodell zur Qualitätsbeurteilung*: eine ordinale Skala, die nicht ein einzelnes Produkt, sondern die *Reife* eines Build-Prozesses bewertet. Damit steht es in einer Reihe mit den etablierten DevSecOps-Reifegradmodellen *OWASP SAMM* [11] und *BSIMM*, die organisatorische und prozessuale Praktiken in Stufen einordnen. Die in der Vorlesung eingeführte Hierarchie aus *Sicherheitspolitik*, *Sicherheitsvorgaben* (Security Target) und *Schutzprofil*

(Protection Profile, vgl. Common Criteria) lässt sich dabei direkt auf die vorliegende Arbeit abbilden: Das Threat-Model (Kap. 4) entspricht der Sicherheitspolitik, die konkrete Workflow-Konfiguration den Sicherheitsvorgaben und eine daraus abgeleitete, repository-unabhängige Hardening-Vorlage einem Schutzprofil.

6.2. OpenSSF Scorecard

Der Scorecard-Workflow läuft wöchentlich. Erwartete Scores $\geq 8/10$ für *Branch-Protection*, *Pinned-Dependencies*, *Token-Permissions*, *Signed-Releases*, *SAST*, *Vulnerabilities*.

6.3. Wirksamkeitsnachweis durch Negativtests

Eine vorhandene Konfiguration belegt noch nicht, dass ein Gate tatsächlich *reagiert*. Zur Plausibilisierung wurden zwei Negativtests am Build- und am SCA-Gate durchgeführt, die jeweils ein reales Defizit blockierten.

Negativtest 1 – Hash-Integrität. Die zuvor manuell gepflegte `requirements.txt` enthielt nicht reproduzierbare Hashes. Der Build-Schritt `pip install -require-hashes` brach daher kontrolliert ab (Auszug):

```
1 ERROR: THESE PACKAGES DO NOT MATCH THE HASHES FROM THE REQUIREMENTS FILE
2 .
3 gunicorn==23.0.0 from .../gunicorn-23.0.0-py3-none-any.whl:
4   Expected sha256
5     ec400d38950a4a6a4f4e2c16a1cd44a30b9b51e9ce2b2b5a3e92c05e4de1d4e8
6   Expected or
7     f014447a0101dc57e294f6c18ca6b40227a4c90e9bdb586042628030cbe004f9
8   Got
9     ec400d38950de4dfd418cff8328b2c8faed0edb0d517d3394e457c317908ca4d
```

Listing 4: Build-Abbruch bei Hash-Diskrepanz

Erst die Neugenerierung des Lockfiles via `pip-compile -generate-hashes` über den vollständigen transitiven Abschluss (neun Pakete) lieferte gegen PyPI verifizierbare Hashes. Der korrigierte Eintrag lautet:

```
1 gunicorn==23.0.0 \
2   --hash=sha256:
3     ec400d38950de4dfd418cff8328b2c8faed0edb0d517d3394e457c317908ca4d
4   \
5   --hash=sha256:
6     f014447a0101dc57e294f6c18ca6b40227a4c90e9bdb586042628030cbe004ec
```

Listing 5: Korrigierter, reproduzierbarer Hash-Eintrag

Danach lief `pip install -require-hashes -no-deps` fehlerfrei durch. Der Test belegt, dass das Gate Reproduzierbarkeit erzwingt und bei jeder Abweichung den Build abbricht – unabhängig davon, ob sie aus einer Lieferketten-Manipulation oder aus fehlerhafter manueller Pflege stammt.

Negativtest 2 – bekannte Schwachstellen (SCA). Gegen dasselbe, hash-verifizierte Lockfile meldete `pip-audit -strict` eine reale Advisory:

```

1 Found 1 known vulnerability in 1 package
2 Name      Version  ID                               Fix Versions
3 -----
4 flask     3.0.3    GHSA-68rp-wp8r-4726             3.1.3

```

Listing 6: pip-audit blockiert auf realer Advisory

Der CI-Job schlägt damit fehl, bis die Abhängigkeit angehoben wird; das SCA-Gate ist also nicht nur vorhanden, sondern reaktiv. Als Konsequenz ist `flask` auf die genannte Fix-Version 3.1.3 anzuheben und das Lockfile (Anhang A.7) neu zu erzeugen.

Methodischer Hinweis. Die Auflösung erfolgte mit Python 3.14, da lokal kein 3.12-Interpreter verfügbar war; für `flask/gunicorn` ist die Marker-Auflösung ab Python 3.10 mit der Laufzeitumgebung (`python:3.12-slim-bookworm`) identisch. Für vollständige Deckungsgleichheit kann das Lockfile im Zielcontainer regeneriert werden.

6.4. Coverage der STRIDE-Kategorien

Tabelle 5 fasst zusammen, welche STRIDE-Kategorien durch die Maßnahmen mitigiert sind.

Kategorie	Mitigiert	Wesentliche Maßnahme
Spoofing	vollständig	OIDC, signierte Commits, Cosign
Tampering	weitgehend	Pinning, Provenance, Verify-Gate
Repudiation	vollständig	Provenance + Rekor-Transparency-Log
Information Disclosure	weitgehend	gitleaks, least-privilege Tokens
Denial of Service	teilweise	concurrency-cancel, Cache-Strategie
Elevation of Privilege	vollständig	default-deny permissions, Environments

Tabelle 5: STRIDE-Coverage durch implementierte Maßnahmen.

6.5. Validierung gegen OWASP Top 10 CI/CD

Zur Plausibilisierung der STRIDE-Analyse wird die abgeleitete Maßnahmenliste zusätzlich gegen die OWASP-Top-10-CI/CD-Risiken [6] geprüft. Tabelle 6 zeigt die Abdeckung.

Nr.	OWASP-Risiko	Wesentliche Maßnahme
1	CICD- Insufficient Flow Control Mechanisms	Branch-Protection, CODEOWNERS, Environments mit Approval
2	CICD- Inadequate Identity & Access Mgmt	OIDC-Federation, least-privilege permissions
3	CICD- Dependency Chain Abuse	Pinning, pip-audit, Dependabot, SBOM

Nr.	OWASP-Risiko	Wesentliche Maßnahme
4	CICD- Poisoned Pipeline Execution (PPE)	Kein <code>pull_request_target</code> , CODEOWNERS auf <code>.github/</code>
5	CICD- Insufficient PBAC	Per-Job <code>permissions</code> , Environment-Reviewer
6	CICD- Insufficient Credential Hygiene	gitleaks, Push-Protection, Cosign keyless (keine Long-lived Keys)
7	CICD- Insecure System Configuration	Scorecard wöchentlich, Action-Pinning, default-deny Permissions
8	CICD- Ungoverned Usage of 3rd-Party Services	Kuratierte Action-Liste, SHA-Pin, Dependabot-Audit
9	CICD- Improper Artifact Integrity Validation	Cosign-Verify + SLSA-Provenance vor Deploy, Digest-Pinning
10	CICD- Insufficient Logging & Visibility	SARIF-Upload, Rekor Transparency-Log, Audit-Logs (org-level)

Tabelle 6: Abdeckung der OWASP-Top-10-CI/CD-Risiken durch die implementierte Pipeline.

Alle zehn Risikoklassen werden adressiert. CICD-10 ist organisatorisch abhängig (Audit-Logs auf Org-Ebene erfordern entsprechende GitHub-Enterprise-Konfiguration) und damit nur teilweise innerhalb der Pipeline selbst mitigierbar.

6.6. KPIs und Sicherheitsmetriken

Die bisherigen Evaluationsschritte (SLSA-Einstufung, Scorecard, STRIDE-Coverage, OWASP-Mapping) bewerten den *strukturellen* Reifegrad der Pipeline – also die Frage, welche Mechanismen implementiert sind. Im laufenden Betrieb muss jedoch zusätzlich beobachtet werden, ob diese Mechanismen tatsächlich *wirken* und welche Reibung sie erzeugen. Dafür werden im Folgenden Kennzahlen definiert, die kontinuierlich aus Pipeline- und Plattform-Telemetrie gewonnen werden können. Die Auswahl orientiert sich am SE-Framework-Quadranten *Assurance* (Tab. 1) und an den DORA-Metriken, ergänzt um sicherheitsspezifische Größen.

Erkennungs- und Reaktionszeiten. *Mean Time to Detect* (MTTD) misst die Zeit von Einführung einer Schwachstelle bis zu ihrer ersten Erkennung durch die Pipeline (etwa über pip-audit, Trivy oder das Rekor-Log). *Mean Time to Remediate* (MTTR) misst die Zeit von Erkennung bis zum gemergten Fix. Beide Größen werden pro Schweregrad (CRITICAL/HIGH/MEDIUM) getrennt geführt; eine MTTR von 24h für CRITICAL ist üblicher Zielwert in modernen DevSecOps-Programmen.

Coverage-Indikatoren. Diese Kennzahlen messen die *Verbreitung* der Schutzmaßnahmen über das Portfolio:

- Anteil signierter Artefakte an allen produzierten Images (Zielwert 100 % für Production-Images),
- Anteil der Repositories mit aktiver Branch-Protection und CODEOWNERS-Datei,

- Anteil der Workflows mit `permissions: {}` als Default (gegen Plattform-Audit-Daten zählbar),
- SAST/SCA-Coverage: Anteil der Code-Pfade, die durch CodeQL bzw. pip-audit tatsächlich analysiert werden.

Reibungsindikatoren. Diese Größen erfassen die in Abschnitt 6.10 diskutierte Anreizlage quantitativ:

- *False-Positive-Rate* der Build-Breaker – der Anteil gefailter Builds, deren Finding nachträglich als nicht-relevant eingestuft wurde. Steigt dieser Wert über ca. 20 %, ist mit systematischem Umgehen der Gates zu rechnen.
- *Pipeline-Bypass-Rate* – Anteil der Merges nach `main`, bei denen mindestens ein Pflicht-Check übersprungen oder von einem Admin überstimmt wurde (in GitHub über das `branches-API` ablesbar).
- *Median-Build-Zeit* für PR-Workflows – ein Anstieg signalisiert, dass die Reibung zunimmt; in Kombination mit der False-Positive-Rate ist dies ein Frühindikator für Umgehungsversuche.

Erweiterte DORA-Metriken. Die klassischen DORA-Größen (Deployment Frequency, Lead Time for Changes, Change Failure Rate, MTTR) lassen sich um eine sicherheits-spezifische Variante ergänzen: die *Security Change Failure Rate* – der Anteil der Deployments, die wegen eines Sicherheitsvorfalls (nicht eines Funktionsdefekts) zurückgerollt werden müssen. Idealerweise konvergiert dieser Wert gegen Null, während die normale Change Failure Rate als Ausdruck gesunder Experimentierkultur durchaus moderate Werte annehmen darf.

Operationalisierung. Im Prototyp werden MTTD/MTTR über die GitHub-Security-Alert-API ausgewertet; Coverage-Indikatoren sind direkt aus der Workflow-Telemetrie und dem Cosign/Rekor-Log ableitbar. Eine vollständige KPI-Erhebung erfordert jedoch eine Org-weite Aggregation, die über GitHub Enterprise Audit Log oder ein dediziertes SIEM erfolgt – diese organisatorische Schicht liegt außerhalb des in Abschnitt 1.3 definierten Scopes. Tabelle 7 fasst die diskutierten Kennzahlen mit Quelle und Zielwert zusammen.

6.7. Compliance und regulatorischer Rahmen

Die in dieser Arbeit umgesetzten Mechanismen erfüllen nicht nur selbstgesetzte Sicherheitsziele, sondern adressieren zugleich externe Anforderungen aus etablierten Standards und EU-Regulierung. Dieser Abschnitt ordnet die Pipeline-Maßnahmen in diese Rahmen ein, ohne ein vollständiges Audit-Mapping zu liefern.

NIST SSDF (SP 800-218). Das *Secure Software Development Framework* [2] strukturiert Praktiken in vier Gruppen: *Prepare the Organization* (PO), *Protect the Software* (PS), *Produce Well-Secured Software* (PW) und *Respond to Vulnerabilities* (RV). Die

Kennzahl	Datenquelle	Zielwert
MTTD (CRITICAL/HIGH)	pip-audit, Trivy, Dependabot	< 24 h
MTTR (CRITICAL)	GitHub Security Alerts	< 7 d
Anteil signierter Images	Cosign / Rekor	100 % (Prod)
Branch-Protection Coverage	GitHub Org-API	100 % (Prod-Repos)
False-Positive-Rate Gates	Build-Logs + Annotation	< 20 %
Pipeline-Bypass-Rate	Branch-API	0 %
Security Change Failure Rate	Deploy-Logs + Incident-Tracker	→ 0

Tabelle 7: Kennzahlen zur kontinuierlichen Bewertung der Pipeline-Sicherheit.

Pipeline adressiert insbesondere PS (durch signierte Artefakte und Provenance), PW (durch SAST/SCA und ein dokumentiertes Threat-Model in Kap. 4) sowie RV (durch Dependabot und Vulnerability-Disclosure via SECURITY.md). Die SSDF-Praktik PW.4 („Reuse Existing, Well-Secured Software“) wird durch Pinning auf SHAs und kuratierte Dependency-Listen unterstützt.

EU Cyber Resilience Act (CRA). Der CRA [7] verpflichtet Hersteller von „Produkten mit digitalen Elementen“ ab 2027 zu SBOM-Bereitstellung, Schwachstellen-Handling über die Produkt-Lebensdauer und 24-Stunden-Reporting bei aktiv ausgenutzten Schwachstellen. Die SBOM-Pflicht ist durch den Lebenszyklus aus Abschnitt 5.5 *by construction* erfüllt; das Reporting ist organisatorisch flankiert durch SECURITY.md und GitHub Private Vulnerability Reporting.

NIS2-Richtlinie. Die NIS2-Richtlinie (EU 2022/2555) verpflichtet „wesentliche“ und „wichtige“ Einrichtungen zu einem Mindestmaß an Cybersicherheits-Maßnahmen, die explizit *Lieferketten-Sicherheit* (Art. 21 Abs. 2 lit. d) und *Sicherheit beim Erwerb, Entwicklung und Wartung von Systemen* (lit. e) umfassen. Die hier umgesetzten Maßnahmen – insbesondere Provenance, signierte Artefakte und Dependency-Tracking – sind eine direkte technische Antwort auf diese beiden Punkte. NIS2 trifft die Studienarbeit-Domäne selbst nicht, ist jedoch der voraussichtlich wirkungsstärkste regulatorische Treiber für DevSecOps in deutschen Unternehmen der nächsten Jahre.

Weitere relevante Rahmen. *ISO/IEC 27001:2022* adressiert in Annex A.8.28 („Secure coding“) und A.8.29 („Security testing in development and acceptance“) Anforderungen, die durch SAST/SCA und das Coverage-Gate operationalisiert werden. *BSI IT-Grundschutz* behandelt CI/CD im Baustein *CON.8 Software-Entwicklung*; die zentralen Anforderungen (kontrollierter Build-Prozess, Schwachstellenmanagement, Schutz der Entwicklungsumgebung) sind durch die Pipeline umgesetzt. Eine vollständige Zertifizierung gegen diese Standards erfordert allerdings organisatorische Belege jenseits des Pipeline-Scopes (Schulungsnachweise, dokumentierte Verantwortlichkeiten, Audit-Reports) und ist daher nicht Gegenstand dieser Arbeit.

Datenschutz und unternehmerische Sorgfaltspflichten. Über die produktbezogene Regulierung hinaus benennt die Aufgabenstellung explizit den datenschutz- und gesellschaftsrechtlichen Rahmen. Die in Tabelle 3 geführte Bedrohung *Deploy-I* (verbose Logs leaken Tokens / PII) berührt unmittelbar die *DSGVO* und das *BDSG*: Personenbezogene Daten in Build-Logs sind eine verarbeitungs- und potenziell meldepflichtige Tatsache, weshalb Log-Maskierung (`add-mask`) und least-privilege-Logging zugleich Sicherheits- und Datenschutzmaßnahmen sind. Begrifflich ist dabei die in der Vorlesung getroffene Unterscheidung relevant: *Geheimhaltung* (technischer Mechanismus), *Vertraulichkeit* (Schutz zugunsten der Organisation) und *Privatsphäre* (Schutz zugunsten des Individuums). Auf der Leitungsebene verpflichtet zudem das *KonTraG* die Unternehmensführung zu einem Risikofrüherkennungssystem; eine auditierbare, mit Provenance und Audit-Logs hinterlegte Pipeline ist insofern auch ein Baustein der gesetzlich geforderten Governance. Das in der Aufgabenstellung ebenfalls genannte *TMG* (Telemediengesetz) ist für die Build-Domäne nur am Rande einschlägig und wurde 2024 weitgehend durch das Digitale-Dienste-Gesetz (DDG) abgelöst; die datenschutzrechtlichen Teile finden sich heute im TDDDG.

Zwischenfazit. Die Pipeline trifft die technischen Anforderungen der relevanten Rahmen (CRA, NIS2, SSDF, ISO 27001 Annex A, BSI CON.8) ohne zusätzliche Sondermaßnahmen. Das ist kein Zufall, sondern Folge der in Kapitel 5 getroffenen Designentscheidungen: Provenance, Signatur und Vulnerability-Tracking sind zugleich Sicherheits- und Compliance-Mechanismen. Diese Doppelfunktion ist im Quadranten *Assurance* (Tab. 1) verortet – Maßnahmen erzeugen sowohl technische Wirksamkeit als auch nachweisbare Belege für externe Stellen.

6.8. Restrisiken

Mehrere Risiken liegen außerhalb der Reichweite der Pipeline. Sie lassen sich mit der in der Vorlesung eingeführten NSA-Unterscheidung präzisieren: Eine Komponente ist *trusted*, wenn ihr Ausfall die Sicherheitspolitik verletzen kann – nicht zwingend, weil sie *trustworthy* (nachweislich nicht-versagend) wäre. GitHub und Sigstore sind in diesem Sinne notwendig *trusted*, aber nicht beweisbar *trustworthy*; genau das macht ihre Kompromittierung zu einem nicht restlos eliminierbaren Restrisiko.

- **Plattform-Kompromittierung** (GitHub, Sigstore): nicht durch die Pipeline mitigierbar; *trusted* Vertrauensanker.
- **Insider mit Admin-Rechten** kann Branch-Protection deaktivieren. Mitigation: Audit-Logging, Org-Policies.
- **Zero-Day in transitiver Abhängigkeit** – pip-audit erkennt nur bekannte CVEs.
- **Secret-Exfiltration via Build-Logs** bei fahrlässigem `echo`; nur durch Code-Review und `add-mask` mitigierbar.

6.9. Zielkonflikt Geschwindigkeit vs. Assurance

Strikte Gates (HIGH/CRITICAL fail, Coverage 80 %, manuelle Approvals) verlangsamen Deployments. Die Pipeline mildert dies durch parallele Jobs, GHA-Caches und `concurrency`:

`cancel-in-progress` für PRs. Production behält bewusst eine Approval-Hürde – Geschwindigkeit darf hier nicht über Assurance dominieren.

6.10. Faktor Mensch und Anreize

Technische Mechanismen entfalten ihre Wirkung nur, wenn die beteiligten Akteure – Entwickler, Reviewer, Plattform-Administratoren – sie auch konsequent anwenden. Anderson formuliert im Security Engineering Framework die These, dass Sicherheit nicht primär an fehlenden Mechanismen scheitert, sondern an fehlerhaft gesetzten *Anreizen*: Die Person, die schützen *soll*, hat oft nicht denselben Nutzen vom erfolgreichen Schutz wie die Person, die im Schadensfall den Verlust trägt. Dieser Anreizbruch ist im DevSecOps-Kontext besonders prägnant, weil das Liefer- und das Sicherheits-Ziel dieselbe Pipeline teilen. Die folgenden Spannungsfelder sind in der Praxis besonders relevant.

Geschwindigkeit vs. Reibung. Sicherheits-Gates wie Coverage-Schwellen, signierte Commits oder Manual-Approval-Schritte verlangsamen den Feedback-Zyklus. Wird die Reibung als unverhältnismäßig empfunden, neigen Teams dazu, Schutzmaßnahmen zu umgehen – etwa durch `[skip ci]`-Commits, `-no-verify` bei pre-commit-Hooks oder das pauschale Akzeptieren von HIGH-Findings als „false positive“. Verhaltensökonomisch lässt sich das mit dem in der Vorlesung diskutierten Modell der *hyperbolischen Diskontierung* beschreiben: Dem unmittelbaren Nutzen (PR ist gemerged, Deployment läuft) wird ein unverhältnismäßig hoher Wert beigemessen gegenüber dem zukünftigen, hypothetischen Nutzen einer vermiedenen CVE-Ausnutzung. Die Pipeline mildert dies, indem sie schnelle Jobs (Lint, Tests) parallel zu langsamen (CodeQL, Trivy) ausführt und PRs bei neuen Commits via `cancel-in-progress` abbricht. Die wichtigere Beobachtung ist jedoch struktureller Natur: Wenn der lokale `pre-commit`-Hook dieselben Checks ausführt wie die CI, fängt ein Entwickler Verstöße bereits vor dem Push – der Schutz wird damit zur *schnellsten* Option, nicht zur langsamsten, und der Diskontierungseffekt arbeitet jetzt zugunsten der Sicherheit.

Ownership-Diffusion. „Security ist Aufgabe des Security-Teams“ ist ein verbreitetes Anti-Pattern; in Conway’s Sinn spiegelt jede Pipeline die Verantwortungsstruktur der Organisation wider, die sie betreibt. Eine zentrale Security-Funktion mit Veto-Recht erzeugt typischerweise eine Zwei-Klassen-Pipeline: schnelle „Dev“-Builds und langsame „Security“-Audits, die sich entkoppelt entwickeln. CODEOWNERS verschiebt das Eigentum für sicherheitskritische Pfade (`.github/`, `Dockerfile`) explizit auf die Pipeline-Verantwortlichen und macht Reviews damit zum unumgehbaren Default; gleichzeitig wird das Reviewer-Recht nicht zentralisiert, sondern an die fachlich nächsten Personen delegiert. Die OpenSSF-Scorecard liefert eine *externe* Bewertung, die Diskussionen über Posture-Defizite versachlicht und sie aus der politischen in die operative Sphäre verschiebt.

Alarmmüdigkeit. Eine Pipeline, die bei jedem Push fünfzig Findings produziert, wird ignoriert – ein Muster, das in der Forschung als *alert fatigue* seit Jahren empirisch belegt ist. Die Implementierung adressiert dies durch die Trennung in Build-Breaker (HIGH, CRITICAL,

mit `ignore-unfixed: true`) und reine Reportings via SARIF. Damit bleibt jeder Build-Abbruch handlungsrelevant, und Findings ohne unmittelbaren Fix verschwinden nicht aus der Sicht (Code-Scanning-View), erzwingen aber keinen Stillstand. Wichtig ist zudem die *Reproduzierbarkeit lokal*: Wenn ein Entwickler denselben Bandit- oder Trivy-Lauf auf der Workstation reproduzieren kann, sinkt die Hemmschwelle, ein Finding tatsächlich zu adressieren, massiv – daher das `pre-commit`-Setup mit denselben Tools.

Anreiz für Maintainer. Dependabot erzeugt Updates als kleine, isolierte PRs, die die volle Pipeline durchlaufen. Damit wird das Aktualisieren von Abhängigkeiten zur *billigsten* statt zur teuersten Option – ein klassischer Anreiz-Hebel im Sinne des Security-Engineering-Frameworks (Tab. 1). Der Hebelpunkt ist subtil: Ohne Automatisierung ist ein Dependency-Update ein bewusst zu treffender Aufwand, der mit dem Risiko eines Regressionsfehlers gegen den Nutzen einer hypothetischen zukünftigen CVE-Vermeidung abgewogen werden muss. Mit Automatisierung kehrt sich das Kosten/Nutzen-Verhältnis um: Den PR *nicht* zu mergen erfordert eine aktive Begründung.

Asymmetrie zwischen Angreifer und Verteidiger. Eine in der Lehrveranstaltung mehrfach betonte Beobachtung ist die *strukturelle Asymmetrie*: Der Angreifer muss eine Lücke finden, der Verteidiger alle schließen. In CI/CD potenziert sich dieser Effekt, weil die Pipeline Vertrauensanker für eine ganze Reihe nachgelagerter Systeme ist – ein einziger Punkt, an dem ein Angreifer transitiv die Produktion kompromittiert. Die Antwort der Pipeline darauf ist nicht Perfektion, sondern Verteidigung in Tiefe: SHA-Pinning (verhindert Tag-Retag), least-privilege Permissions (begrenzt Sprengkraft eines erfolgreichen PPE), Cosign-Verify (erkennt Tag-Hijack post-build), Manual-Approval (letzte menschliche Hürde). Jede Einzelmaßnahme ist umgehbar; die Kombination ist es deutlich weniger.

Verantwortungs-Verschiebung durch „Shift Left“. Der DevSecOps-Slogan „shift left“ verschiebt Sicherheitsverantwortung von einem nachgelagerten Audit-Team zu den Entwicklern selbst. Das ist intendierter Effekt – Bugs werden früher und billiger behoben – erzeugt aber eine Kehrseite: Entwickler werden für Entscheidungen verantwortlich, für die sie weder ausgebildet sind noch die Zeit zugewiesen bekommen. Ein Bandit-Finding zu einer `subprocess.shell=True`-Verwendung erfordert eine Sicherheitsbewertung im Kontext der Aufrufkette. Werden solche Findings ohne unterstützendes Material allein dem Entwickler überlassen, ist die Standardreaktion das Stummschalten per `# nosec`-Kommentar. Die Pipeline kompensiert das nur teilweise (Bandit-Findings landen als SARIF im Code-Scanning-View und sind damit auch für andere Reviewer sichtbar) – die verbleibende Schulungs- und Begleitarbeit ist eine organisatorische Investition, die kein Tool ersetzt.

Falsche Sicherheit durch Zertifikate. Eine grüne Pipeline und ein hoher Scorecard-Wert können den Eindruck erwecken, ein Repository sei „sicher“. Schramm verweist in der Vorlesung explizit auf das Phänomen *Security Theatre* (nach Schneier): sichtbare, aber wirkungsschwache Maßnahmen werden wirksamen, aber unsichtbaren vorgezogen, weil sie das Vertrauen der Stakeholder herstellen. Strukturell verwandt ist das in der Vorlesung behandelte *Datenschutzparadoxon* – die empirisch beobachtete Lücke zwischen geäußelter Sicherheits-/Datenschutz-Präferenz und tatsächlichem Verhalten. Im DevSecOps-Kontext

zeigt sich dasselbe Muster auf Team-Ebene: In Retrospektiven und Architektur-Reviews wird Sicherheit als hoher Wert benannt; im Tagesgeschäft setzen sich jedoch Geschwindigkeit und Feature-Lieferung durch, sobald das nominale Schutzniveau *erscheint* ausreichend zu sein. Die OpenSSF-Scorecard ist anfällig für genau dieses Muster, da sie objektiv messbare Indikatoren bewertet (Existenz einer SECURITY.md, Action-Pinning, Branch-Protection-Konfiguration), nicht aber die *Qualität* der gelebten Praxis: Ob die SECURITY.md tatsächlich gelesen wurde, ob CODEOWNERS-Reviews substanziell oder rubber-stamp sind, ob die Build-Breaker-Findings wirklich behoben oder nur weggelencet werden – all das fällt durch das Raster. Die Pipeline ist daher nicht als Endpunkt, sondern als *Mindeststandard* zu lesen, auf dem eine kontinuierliche Sicherheitskultur aufsetzen muss.

Insider als Grenzfall. Alle hier diskutierten Mechanismen unterstellen, dass die Pipeline-Konfiguration selbst durch reguläre Reviews geschützt ist. Ein Administrator mit Force-Push-Recht oder das Recht, Branch-Protection zu deaktivieren, kann das gesamte Schutzgerüst aushebeln, ohne dass eine einzige technische Komponente versagt. Das ist kein Defekt der Pipeline, sondern eine fundamentale Eigenschaft jedes Systems mit unteilbarer Admin-Identität. Die Mitigation liegt außerhalb der Pipeline: rollengetrennte Org-Strukturen, Audit-Logging mit unabhängigem Empfänger, regelmäßige Posture-Reviews durch externe Stellen. Diese Maßnahmen werden in den Restrisiken (Abschnitt 6.8) als bewusst nicht durch die Pipeline adressierbar geführt.

Zusammenfassung. Die diskutierten Spannungsfelder zeigen, dass die untersuchte Pipeline *nicht* nur ein Werkzeug-Stapel ist, sondern eine Antwort auf eine Anreizlage: Sie macht Sicherheit zur billigsten verfügbaren Option, sie verteilt Verantwortung über die Organisation, sie entkoppelt Build-Breaker von Reportings, und sie hält bewusst eine menschliche Hürde an der kritischsten Stelle (Production-Approval) aufrecht. Diese Entscheidungen sind in den Quadranten *Incentives* des SE-Frameworks (Tab. 1) verortet und damit gleichberechtigt zu den technischen Mechanismen.

7. Fazit und Ausblick

Die Arbeit zeigt, dass sich eine moderne CI/CD-Pipeline mit zugänglichen Mitteln (GitHub Actions, Sigstore, Trivy, Syft) systematisch entlang einer STRIDE-Analyse absichern lässt und dabei nachweisbare Eigenschaften gemäß SLSA erreicht. Wesentlich ist nicht das einzelne Tool, sondern die Kombination: *least-privilege Identitäten, gepinnte Abhängigkeiten, signierte Artefakte mit Provenance, verifizierte Deployments*.

Weiterführende Arbeiten könnten:

- die Übertragbarkeit der Maßnahmen auf GitLab CI quantitativ vergleichen,
- Policy-as-Code (z. B. Kyverno, Sigstore Policy Controller im Cluster) als zusätzliche Verifikationsschicht ergänzen,
- formale Analysen zu Pipeline-Konfigurationen (z. B. zizmor, actionlint) mit Beweisbarkeit der *permissions*-Minimalität verbinden,

- den SBOM-Lebenszyklus um den in Abschnitt 5.5 skizzierten kontinuierlichen SBOM-Diff erweitern und das SBOM um eine *Cryptography Bill of Materials* (CBOM) ergänzen, die kryptographische Assets inventarisiert – relevant insbesondere für eine spätere Post-Quantum-Migration.

Literatur

- [1] OpenSSF, *Supply-chain Levels for Software Artifacts (SLSA) v1.0*, <https://slsa.dev/spec/v1.0/>, abgerufen 2026.
- [2] NIST, *Secure Software Development Framework (SSDF) Version 1.1*, SP 800-218, 2022.
- [3] Microsoft, *The STRIDE Threat Model*, Microsoft Learn, abgerufen 2026.
- [4] OpenSSF, *Scorecard*, <https://github.com/ossf/scorecard>, abgerufen 2026.
- [5] Sigstore Project, *Cosign / Fulcio / Rekor*, <https://www.sigstore.dev/>, abgerufen 2026.
- [6] OWASP, *Top 10 CI/CD Security Risks*, <https://owasp.org/www-project-top-10-ci-cd-security-risks/>, abgerufen 2026.
- [7] Europäische Union, *Cyber Resilience Act* (Verordnung (EU) 2024/2847), <https://eur-lex.europa.eu/eli/reg/2024/2847/oj>, 2024.
- [8] Ross J. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3. Aufl., Wiley, 2020.
- [9] D.E. Bell, L.J. LaPadula, *Secure Computer Systems: Mathematical Foundations*, MITRE Technical Report 2547, 1973.
- [10] K. J. Biba, *Integrity Considerations for Secure Computer Systems*, MITRE Technical Report 3153, 1977.
- [11] OWASP, *Software Assurance Maturity Model (SAMM) v2*, <https://owaspsamm.org/>, abgerufen 2026.

A. Vollständige Pipeline-Artefakte

Die im Fließtext (Kap. 5) gezeigten Auszüge sind aus Platzgründen gekürzt. Dieser Anhang dokumentiert die im Repository tatsächlich versionierten Artefakte vollständig und dient als Beleg für die in den Kapiteln 5 und 6 beschriebenen Eigenschaften: durchgängiges Pinning aller Actions auf Commit-SHAs, Least-Privilege-permissions je Job sowie das Verifikations-Gate (Signatur + SLSA-Provenance) vor jedem Deploy.

A.1. CI-Workflow (`.github/workflows/ci.yml`)

```

1 # Hardened CI pipeline: lint, test, SAST, SCA, SBOM, container scan,
  signing.
2 # Design notes:
3 # * Runs on PRs and pushes to main only; tags trigger the release flow.
4 # * Every job pins actions to commit SHAs to defeat tag-retag attacks.
5 # * Uses GITHUB_TOKEN with least-privilege "permissions" per job.
6 # * Egress is implicitly limited via dependency caches + pinned indexes
  .
7 name: CI
8
9 on:
10   push:
11     branches: [main]
12   pull_request:
13     branches: [main]
14
15 permissions: {} # default-deny; jobs opt in below.
16
17 concurrency:
18   group: ci-`${{ github.ref }}`
19   cancel-in-progress: true
20
21 env:
22   PYTHON_VERSION: "3.12"
23
24 jobs:
25   lint-test:
26     name: Lint, Unit Tests, Coverage
27     runs-on: ubuntu-24.04
28     permissions:
29       contents: read
30     steps:
31       - name: Checkout
32         uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
33         # v4.2.2
34         with:
35           persist-credentials: false
36
37       - name: Setup Python
38         uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5.6.0
39         with:
40           python-version: `${{ env.PYTHON_VERSION }}`
41           cache: pip
42
43       - name: Install deps
44         run: |
45           python -m pip install --upgrade pip
46           pip install -r app/requirements.txt -r app/requirements-dev.
47             txt
48
49       - name: Ruff (lint)
50         run: ruff check app
51
52       - name: Pytest with coverage
53         run: pytest --cov=app --cov-report=xml --cov-fail-under=80 app/
54           tests

```



```

52     - name: Upload coverage artifact
53       uses: actions/upload-
54         artifact@ea165f8d65b6e75b540449e92b4886f43607fa02 # v4.6.2
55       with:
56         name: coverage
57         path: coverage.xml
58         retention-days: 7
59
60   sast:
61     name: SAST (Bandit + CodeQL)
62     runs-on: ubuntu-24.04
63     permissions:
64       contents: read
65       security-events: write
66     steps:
67       - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
68         # v4.2.2
69       with:
70         persist-credentials: false
71
72       - name: Setup Python
73         uses: actions/setup-
74           python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5.6.0
75       with:
76         python-version: ${ env.PYTHON_VERSION }
77
78       - name: Bandit
79         run: |
80           pip install bandit==1.7.10
81           bandit -r app -ll -ii -f sarif -o bandit.sarif || true
82
83       - name: Upload Bandit SARIF
84         uses: github/codeql-action/upload-
85           sarif@4e94bd11f71e507f7f87df81788dff88d1dacbfb # v3.28.10
86       with:
87         sarif_file: bandit.sarif
88         category: bandit
89
90       - name: Initialize CodeQL
91         uses: github/codeql-action/
92           init@4e94bd11f71e507f7f87df81788dff88d1dacbfb # v3.28.10
93       with:
94         languages: python
95
96       - name: Perform CodeQL Analysis
97         uses: github/codeql-action/
98           analyze@4e94bd11f71e507f7f87df81788dff88d1dacbfb # v3.28.10
99       with:
100         category: codeql
101
102   sca:
103     name: SCA (pip-audit + OSV)
104     runs-on: ubuntu-24.04
105     permissions:
106       contents: read
107     steps:

```

```

103     - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
104       # v4.2.2
105     with:
106       persist-credentials: false
107
108     - name: Setup Python
109       uses: actions/setup-
110         python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5.6.0
111     with:
112       python-version: ${ env.PYTHON_VERSION }
113
114     - name: pip-audit
115       run: |
116         pip install pip-audit==2.7.3
117         pip-audit -r app/requirements.txt --strict --progress-spinner=
118           off
119
120   secret-scan:
121     name: Secret Scan (gitleaks)
122     runs-on: ubuntu-24.04
123     permissions:
124       contents: read
125     steps:
126     - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
127       # v4.2.2
128     with:
129       fetch-depth: 0
130       persist-credentials: false
131
132     - name: Run gitleaks
133       uses: gitleaks/gitleaks-
134         action@ff98106e4c7b2bc287b24eaf42907196329070c7 # v2.3.7
135     env:
136       GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
137       GITLEAKS_ENABLE_UPLOAD_ARTIFACT: "true"
138
139   build-image:
140     name: Build, SBOM, Scan, Sign
141     needs: [lint-test, sast, sca, secret-scan]
142     runs-on: ubuntu-24.04
143     permissions:
144       contents: read
145       packages: write
146       id-token: write # OIDC for cosign keyless
147       attestations: write
148     env:
149       REGISTRY: ghcr.io
150       IMAGE_NAME: ${ github.repository }
151     steps:
152     - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
153       # v4.2.2
154     with:
155       persist-credentials: false
156
157     - name: Set up Docker Buildx
158       uses: docker/setup-buildx-
159         action@e468171a9de216ec08956ac3ada2f0791b6bd435 # v3.11.1

```

```

154 - name: Login to GHCR
155   if: github.event_name != 'pull_request'
156   uses: docker/login-
157     action@74a5d142397b4f367a81961eba4e8cd7edddf772 # v3.4.0
158   with:
159     registry: ${ env.REGISTRY }
160     username: ${ github.actor }
161     password: ${ secrets.GITHUB_TOKEN }
162
163 - name: Extract metadata
164   id: meta
165   uses: docker/metadata-
166     action@902fa8ec7d6ecbf8d84d538b9b233a880e428804 # v5.7.0
167   with:
168     images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
169     tags: |
170       type=sha,prefix=sha-
171       type=ref,event=branch
172       type=ref,event=pr
173       type=semver,pattern={{version}}
174
175 - name: Build (load for scan)
176   uses: docker/build-push-
177     action@263435318d21b8e681c14492fe198d362a7d2c83 # v6.18.0
178   with:
179     context: .
180     file: ./Dockerfile
181     load: true
182     tags: local/scan:${ github.sha }
183     cache-from: type=gha
184     cache-to: type=gha,mode=max
185
186 - name: Trivy image scan (fail on HIGH/CRITICAL)
187   uses: aquasecurity/trivy-
188     action@dc5a429b52fcf669ce959baa2c2dd26090d2a6c4 # v0.32.0
189   with:
190     image-ref: local/scan:${ github.sha }
191     format: sarif
192     output: trivy.sarif
193     severity: HIGH,CRITICAL
194     exit-code: "1"
195     ignore-unfixed: true
196
197 - name: Upload Trivy SARIF
198   if: always()
199   uses: github/codeql-action/upload-
200     sarif@4e94bd11f71e507f7f87df81788dff88d1dacbfbb # v3.28.10
201   with:
202     sarif_file: trivy.sarif
203     category: trivy
204
205 - name: Generate SBOM (CycloneDX) with Syft
206   uses: anchore/sbom-
207     action@cee1b8e05ae5b2593a75e197229729eabaa9f8ec # v0.20.2
208   with:
209     image: local/scan:${ github.sha }
210     format: cyclonedx-json
211     output-file: sbom.cdx.json

```

```

206         upload-artifact: true
207
208     - name: Push image
209       id: push
210       if: github.event_name != 'pull_request'
211       uses: docker/build-push-
212         action@263435318d21b8e681c14492fe198d362a7d2c83 # v6.18.0
213       with:
214         context: .
215         file: ./Dockerfile
216         push: true
217         tags: ${{ steps.meta.outputs.tags }}
218         labels: ${{ steps.meta.outputs.labels }}
219         cache-from: type=gha
220
221     - name: Install cosign
222       if: github.event_name != 'pull_request'
223       uses: sigstore/cosign-
224         installer@d58896d6a1865668819e1d91763c7751a165e159 # v3.9.2
225
226     - name: Sign image (keyless via OIDC)
227       if: github.event_name != 'pull_request'
228       env:
229         DIGEST: ${{ steps.push.outputs.digest }}
230         TAGS: ${{ steps.meta.outputs.tags }}
231         COSIGN_EXPERIMENTAL: "1"
232       run: |
233         for tag in $TAGS; do
234           cosign sign --yes "${tag}@${DIGEST}"
235         done
236
237     - name: Attest SBOM
238       if: github.event_name != 'pull_request'
239       uses: actions/attest-
240         sbom@bd218ad0dbcb3e146bd073d1d9c6d78e08aa8a0b # v2.4.0
241       with:
242         subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
243         subject-digest: ${{ steps.push.outputs.digest }}
244         sbom-path: sbom.cdx.json
245         push-to-registry: true
246
247     - name: Build provenance attestation (SLSA)
248       if: github.event_name != 'pull_request'
249       uses: actions/attest-build-
250         provenance@e8998f949152b193b063cb0ec769d69d929409be # v2.4.0
251       with:
252         subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
253         subject-digest: ${{ steps.push.outputs.digest }}
254         push-to-registry: true

```

A.2. CD-Workflow (.github/workflows/cd.yml)

```

1 # Deploy flow: triggered by signed git tags. Verifies cosign signature
2 # and provenance before promoting through staging -> production via
3 # GitHub Environments (manual approval gate on production).
4 #

```

```

5 # Security invariant: deployments always reference an immutable image
6 # digest (@sha256:...), never a mutable tag. The tag-triggered path
7 # resolves the digest from the registry before handing off to deploy
  jobs.
8 name: CD
9
10 on:
11   push:
12     tags: ["v*.*.*"]
13   workflow_dispatch:
14     inputs:
15       image_digest:
16         description: "Image digest to deploy (sha256:...)"
17         required: true
18
19 permissions: {}
20
21 concurrency:
22   group: cd-`${{ github.ref }}`
23   cancel-in-progress: false
24
25 env:
26   REGISTRY: ghcr.io
27   IMAGE_NAME: `${{ github.repository }}`
28
29 jobs:
30   verify:
31     name: Verify signature & provenance
32     runs-on: ubuntu-24.04
33     permissions:
34       contents: read
35       packages: read
36       id-token: write
37     outputs:
38       digest: `${{ steps.resolve.outputs.digest }}`
39     steps:
40       - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
41         # v4.2.2
42       with:
43         persist-credentials: false
44
45       - name: Install cosign
46         uses: sigstore/cosign-
47           installer@d58896d6a1865668819e1d91763c7751a165e159 # v3.9.2
48
49       - name: Resolve image digest
50         id: resolve
51         env:
52           INPUT_DIGEST: `${{ inputs.image_digest }}`
53           REF_NAME: `${{ github.ref_name }}`
54           REGISTRY: `${{ env.REGISTRY }}`
55           IMAGE_NAME: `${{ env.IMAGE_NAME }}`
56         run: |
57           if [ -n "$INPUT_DIGEST" ]; then
58             # workflow_dispatch: digest provided directly
59             DIGEST="$INPUT_DIGEST"
60           else

```

```

59         # tag push: resolve the digest for the pushed tag so we
        never
60         # deploy by mutable tag -- only by immutable digest.
61         TAGGED_REF="${REGISTRY}/${IMAGE_NAME}:${REF_NAME}"
62         DIGEST=$(docker buildx imagetools inspect "$TAGGED_REF" \
63             --format '{{json .Manifest.Digest}}' | tr -d '"')
64         if [ -z "$DIGEST" ]; then
65             echo "::error::Could not resolve digest for ${TAGGED_REF}"
66             exit 1
67         fi
68     fi
69     echo "digest=${DIGEST}" >> "$GITHUB_OUTPUT"
70     echo "ref=${REGISTRY}/${IMAGE_NAME}@${DIGEST}" >> "
        $GITHUB_OUTPUT"
71     echo "Resolved to digest: ${DIGEST}"
72
73     - name: Verify cosign signature (keyless)
74     env:
75         IMAGE: ${ steps.resolve.outputs.ref }
76     run: |
77         cosign verify \
78             --certificate-identity-regexp "https://github.com/${
                GITHUB_REPOSITORY}/.github/workflows/.+" \
79             --certificate-oidc-issuer "https://token.actions.
                githubusercontent.com" "$IMAGE"
80
81     - name: Verify build provenance attestation
82     env:
83         IMAGE: ${ steps.resolve.outputs.ref }
84     run: |
85         cosign verify-attestation \
86             --type slsaprovenance \
87             --certificate-identity-regexp "https://github.com/${
                GITHUB_REPOSITORY}/.github/workflows/.+" \
88             --certificate-oidc-issuer "https://token.actions.
                githubusercontent.com" \
89             "$IMAGE"
90
91     deploy-staging:
92     name: Deploy to staging
93     needs: verify
94     runs-on: ubuntu-24.04
95     environment:
96     name: staging
97     url: https://staging.example.invalid
98     permissions:
99     contents: read
100    id-token: write    # for cloud OIDC federation (e.g. Azure/AWS)
101    steps:
102    - name: Federated login (placeholder)
103      run: echo "Use azure/login@... or aws-actions/configure-aws-
        credentials@... with OIDC here."
104
105    - name: Deploy (digest-pinned)
106    env:
107        DIGEST: ${ needs.verify.outputs.digest }
108        REGISTRY: ${ env.REGISTRY }
109        IMAGE_NAME: ${ env.IMAGE_NAME }

```

```

110     run: |
111         # Always deploy by digest, never by tag.
112         echo "kubectl set image deploy/app app=${REGISTRY}/${
113             IMAGE_NAME}@${DIGEST}"
114
115     deploy-production:
116         name: Deploy to production (manual approval)
117         needs: [verify, deploy-staging]
118         runs-on: ubuntu-24.04
119         environment:
120             name: production
121             url: https://app.example.invalid
122         permissions:
123             contents: read
124             id-token: write
125         steps:
126             - name: Federated login (placeholder)
127               run: echo "OIDC federation to cloud provider goes here -- no
128                   long-lived secrets."
129
130             - name: Deploy (digest-pinned)
131               env:
132                 DIGEST: ${ needs.verify.outputs.digest }
133                 REGISTRY: ${ env.REGISTRY }
134                 IMAGE_NAME: ${ env.IMAGE_NAME }
135               run: |
136                 # Always deploy by digest, never by tag.
137                 echo "kubectl set image deploy/app app=${REGISTRY}/${
138                     IMAGE_NAME}@${DIGEST}"

```

A.3. Scorecard-Workflow (.github/workflows/scorecard.yml)

```

1  # OpenSSF Scorecard -- continuously evaluates repo posture (branch
2  # protection, signed releases, dangerous workflows, etc.).
3  name: Scorecard
4
5  on:
6    branch_protection_rule:
7    schedule:
8      - cron: "23 4 * * 1"
9    push:
10      branches: [main]
11
12  permissions: {}
13
14  jobs:
15    analysis:
16      name: Scorecard analysis
17      runs-on: ubuntu-24.04
18      permissions:
19        security-events: write
20        id-token: write
21        contents: read
22        actions: read
23      steps:

```

```

24     - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      # v4.2.2
25     with:
26         persist-credentials: false
27
28     - uses: ossf/scorecard-
      action@05b42c624433fc40578a4040d5cf5e36ddca8cde # v2.4.2
29     with:
30         results_file: results.sarif
31         results_format: sarif
32         publish_results: true
33
34     - uses: github/codeql-action/upload-
      sarif@4e94bd11f71e507f7f87df81788dff88d1dacbf # v3.28.10
35     with:
36         sarif_file: results.sarif
37         category: scorecard

```

A.4. Dockerfile (Multi-Stage, Non-Root)

```

1  # syntax=docker/dockerfile:1.7
2  # Multi-stage, distroless-style build.
3  # requirements.txt is generated by pip-compile --generate-hashes so the
4  # --require-hashes flag below is always satisfied. Regenerate with:
5  #   pip-compile --generate-hashes app/requirements.in -o app/
      requirements.txt
6  #
7  # Base image: pin to a digest for full supply-chain integrity.
8  # Update the digest when upgrading Python or applying security patches:
9  #   docker pull python:3.12-slim-bookworm
10 #   docker inspect python:3.12-slim-bookworm --format '{{index .
      RepoDigests 0}}'
11 ARG PYTHON_VERSION=3.12-slim-bookworm
12 # sha256 below is the digest for 3.12-slim-bookworm as of the last
      review.
13 # Replace with the output of the inspect command above before production
      use.
14 ARG PYTHON_DIGEST=sha256:
      e9b6a30f2b3ded7ad1b94e7c5f88492bea44c2f17ee09c0c16c2a6db5e7db1d4
15
16 FROM python:${PYTHON_VERSION}@${PYTHON_DIGEST} AS builder
17 WORKDIR /build
18 ENV PIP_NO_CACHE_DIR=1 \
19     PIP_DISABLE_PIP_VERSION_CHECK=1 \
20     PYTHONDONTWRITEBYTECODE=1
21 COPY app/requirements.txt ./requirements.txt
22 # --require-hashes: fail the build if any wheel/sdist is not hash-pinned
      .
23 # --no-deps: transitives must be listed explicitly in requirements.txt
24 #           (pip-compile --generate-hashes resolves them all).
25 # No silent fallback -- if this fails, regenerate requirements.txt.
26 RUN python -m venv /venv \
27     && /venv/bin/pip install --require-hashes --no-deps -r requirements.
      txt
28
29 FROM python:${PYTHON_VERSION}@${PYTHON_DIGEST} AS runtime

```



```

30 # Drop privileges: run as non-root, read-only filesystem-friendly layout
31 RUN groupadd --system --gid 10001 app \
32     && useradd --system --uid 10001 --gid app --no-create-home --shell
33     /usr/sbin/nologin app
34 WORKDIR /srv/app
35 COPY --from=builder /venv /venv
36 COPY app/ /srv/app/app/
37 ENV PATH="/venv/bin:$PATH" \
38     PYTHONUNBUFFERED=1 \
39     PYTHONDONTWRITEBYTECODE=1 \
40     APP_VERSION=dev
41 USER 10001:10001
42 EXPOSE 8000
43 HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
44     CMD python -c "import urllib.request,sys; sys.exit(0 if urllib.
45         request.urlopen('http://127.0.0.1:8000/healthz').status==200 else
46         1)"
47 ENTRYPOINT ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "app
48     .app:app"]

```

A.5. .github/CODEOWNERS

```

1 # Pipeline-touching changes need both authors -- supply-chain critical.
2 /.github/ @christof-renner @manuel-friedl
3 /Dockerfile @christof-renner @manuel-friedl

```

A.6. Dependabot (.github/dependabot.yml)

```

1 version: 2
2 updates:
3   - package-ecosystem: pip
4     directory: /app
5     schedule:
6       interval: weekly
7       open-pull-requests-limit: 10
8       labels: ["dependencies", "security"]
9   - package-ecosystem: docker
10    directory: /
11    schedule:
12      interval: weekly
13      labels: ["dependencies", "container"]
14   - package-ecosystem: github-actions
15     directory: /
16     schedule:
17       interval: weekly
18     labels: ["dependencies", "ci"]

```

A.7. Abhängigkeits-Definition (app/requirements.in)

Die direkten Laufzeit-Abhängigkeiten. Die vollständige, hash-gepinnte `requirements.txt` (neun Pakete inklusive transitivem Abschluss, 131 Zeilen) wird hieraus mit `pip-compile-generate-hashes` erzeugt; ein repräsentativer Eintrag findet sich in Listing 5.

```
1 flask==3.0.3
2 gunicorn==23.0.0
```

Listing 7: Direkte Abhängigkeiten (Lockfile-Quelle)